

Handy Parking Documentation

V. Marianciuc, V. Natalevych, V. Hurko, D. Hrosh

Engineering team project

Table 1: Versions and changes

Version	Date	Author	Description
0.0.0	24.04.2025	Vitalii Natalevych	Initialization of documentation, added Introduction withou SWOT Analysys, Added Requirement Specifications
0.1.0	24.04.2025	Vladimir Marianciuc	Added Installation process and API Documentation Appendicies

Abstract

This documentation outlines the design, functionality, and implementation of an intelligent Parking Management System developed to streamline vehicle access control and optimize parking operations. The system integrates automatic license plate recognition (ALPR) with gate control mechanisms to enable seamless entry and exit for authorized vehicles. It is designed to enhance security, improve traffic flow, and reduce manual intervention in parking facilities. Key components of the system include vehicle detection and number recognition, real-time database verification, automated gate operation, and a centralized dashboard for monitoring and management. This project serves as a modular and scalable solution, suitable for residential complexes, corporate campuses, and commercial parking lots. The documentation covers system architecture, setup instructions, usage guidelines, and maintenance procedures.

Contents

1	Introduction	6
1.1	Project overview	6
1.2	Key Features	6
1.3	Target Audience	6
1.4	SWOT Analysis	6
1.5	Technology Stack	7
1.5.1	Backend	7
1.5.2	Frontend	8
1.5.3	Mobile Application	8
1.5.4	Parking Gate Controller	9
1.5.5	Integrations	9
2	Requirements Specification	9
2.1	General Requirements	9
2.2	Functional Requirements	10
2.3	Non-functional Requirements	10
3	Application Architecture Design	11
3.1	Overall Architecture Concept	11
3.2	Project Structure	11
3.3	Application Modules	11
3.3.1	Application for Parking Owners	11
3.3.2	Mobile application for Drivers	11
3.3.3	Admin Dashboard	11
3.3.4	Parking Gate Controller	11
3.3.5	Payment Page for unregistered drivers	11
3.4	User Roles	11
4	Project Implementation	11
4.1	Setup & Run Instructions	11
5	Testing	12
5.1	Types of tests	12
5.2	Running test	12
5.3	Testing Frameworks	12
6	Security	12
6.1	Auth & Authentication	12
6.2	Data Protection	12
6.3	Common Vulnerabilities	12
7	Database	12
7.1	Schema Overview	12
7.2	Migrations & Seeding	12
8	Summary	12
9	Appendices	12
9.1	Appendix A: Instalation process	12
9.2	API Documentation	38
9.2.1	Appendix B: Stripe Integration Documentation	38
9.2.2	Appendix C: Email Verification	45
9.2.3	Appendix D: Drivers Manipulations	52
9.2.4	Appendix E: Users Manipulations	60
9.2.5	Appendix F: JWT Public Key	68
9.2.6	Appendix G: Notifications	74
9.2.7	Appendix H : Vehicle Management	83
9.2.8	Appendix I: Vehicle Ownership Management	91
9.2.9	Appendix J: Parking Owners Management	98
9.2.10	Appendix K: Parking Registration Process	107

9.2.11	Appendix L: Reviews	113
9.2.12	Appendix M: Registration and Refresh Token	121
9.2.13	Appendix N: API Keys Management Service	129
9.2.14	Appendix O: Parking Management System API Documentation	138
9.2.15	Appendix P: Authorization and Authentication System Documentation . .	146
9.3	Demo of applications	154
9.3.1	Web Dashboard for Parking owner	154
9.3.2	Mobile Application For Drivers	154
9.3.3	Admin Panel	154
9.3.4	web-page for non-users mobile app	154
9.3.5	Parking Gate Controller	154

1 Introduction

1.1 Project overview

The Parking Management System is a comprehensive solution designed to streamline parking operations and enhance user experience. It consists of multiple integrated components that serve different stakeholders in the parking ecosystem.

1.2 Key Features

- **Parking Owner Portal:** Complete management of parking facilities, including capacity control, tariff setting, and revenue tracking
- **Mobile Application for Drivers:** Convenient parking discovery, payment, and account management
- **Web Payment Option:** Alternative payment method for non-app users
- **Administrative Dashboard:** Central system management and moderation tools
- **Parking Gate Controller:** Automated vehicle recognition and gate operation

1.3 Target Audience

The system serves three primary user groups:

1. **Parking Owners/Operators:** Businesses or individuals who manage parking facilities
2. **Drivers/Parking Users:** Individuals seeking parking spaces and looking to manage their parking sessions
3. **System Administrators:** Personnel responsible for overseeing the entire system and supporting other users

Each component of the system is tailored to meet the specific needs of its target audience while maintaining seamless integration with the other parts of the ecosystem.

1.4 SWOT Analysis

TODO:
SWOT
Analysis

1.5 Technology Stack

1.5.1 Backend

Table 2: Backend technology stack

Technology	Short Description
Java	A robust and widely-used object-oriented programming language known for its platform independence and extensive ecosystem.
Spring	A comprehensive application development framework for Java, providing infrastructure support for building enterprise-level applications.
PostgreSQL	A powerful, open-source relational database system known for its reliability, feature richness, and extensibility.
Kafka	A distributed streaming platform designed for building real-time data pipelines and streaming applications.
Microservice Architecture	An architectural style that structures an application as a collection of small, independent services, each addressing a specific business capability.
Kubernetes	A container orchestration platform for automating deployment, scaling, and management of containerized applications.
Docker	A platform for developing, shipping, and running applications in isolated environments called containers.
Spring Cloud	A framework built on top of Spring that provides tools for building distributed systems and microservices in the cloud.
Spring Security	A powerful and customizable authentication and authorization framework for Java-based applications.
Eureka	A service discovery server and client for locating services within a microservices architecture.
Config Service (Spring Cloud Config)	A centralized configuration management service for distributed applications, allowing externalization of application configurations.
Gateway (Spring Cloud Gateway)	An intelligent and programmable router that acts as a single entry point for client requests in a microservices architecture.
JWT (JSON Web Tokens)	A compact and self-contained way for securely transmitting information between parties as a JSON object.
Zipkin	A distributed tracing system that helps in monitoring and troubleshooting latency issues in microservices environments.

1.5.2 Frontend

Table 3: Frontend technology stack

Technology	Short Description
React	A popular JavaScript library for building user interfaces or UI components.
Typescript	A superset of JavaScript that adds static typing to the language, improving code maintainability and reducing errors.
Vite	A next-generation frontend tooling that provides an extremely fast development experience and bundles code efficiently.
TailwindCSS	A utility-first CSS framework that allows for rapid styling directly in HTML using pre-defined classes.
shadcn/ui	A collection of reusable UI components built using Radix UI and Tailwind CSS, focusing on accessibility and customization.
Axios HTTP	A promise-based HTTP client for browsers and Node.js, making it easy to perform API requests.
Lucide	A collection of beautiful and consistent open-source icons.
MobX	A simple and scalable state management library that makes it easy to connect the data of your application with the UI.
Zod	A TypeScript-first schema declaration and validation library that allows you to define and verify data structures.
Next.js	A popular React framework for building full-stack web applications with features like server-side rendering and routing.
HTML	HyperText Markup Language, the standard markup language for creating web pages and web applications. It provides the structure and content of a webpage.
CSS	Cascading Style Sheets, a stylesheet language used to describe the presentation of a document written in a markup language like HTML. It controls the layout, colors, fonts, and other visual aspects of a website.

1.5.3 Mobile Application

Table 4: Mobile Application technology stack

Technology	Short Description
React Native	A framework for building native mobile applications using JavaScript and React. It allows developers to write code once and deploy it on both iOS and Android.
Typescript	A superset of JavaScript that adds static typing to the language, improving code maintainability and reducing errors, and is often used with React Native for mobile development.
Expo	A framework and platform built on top of React Native that simplifies the development, building, deployment, and updating of native iOS and Android applications.
Kotlin	A modern, statically-typed programming language for the JVM, Android, and the browser, known for its conciseness and interoperability with Java. It is a primary language for Android development.
Swift	A powerful and intuitive programming language developed by Apple for building apps across all Apple platforms (iOS, macOS, watchOS, tvOS, and beyond). It is the primary language for native iOS development.
Axios	A promise-based HTTP client for browsers and Node.js, commonly used in mobile development with frameworks like React Native to make API requests.

1.5.4 Parking Gate Controller

Table 5: Parking Gate Controller technology stack

Technology	Short Description
Python	A versatile and widely-used high-level programming language known for its readability and extensive libraries, making it popular in areas like data science, web development, and machine learning.
OpenCV	An open-source computer vision library providing a wide range of algorithms for image and video processing, analysis, and computer vision tasks.
PaddleOCR	An open-source Optical Character Recognition (OCR) system developed by Baidu, supporting multiple languages and providing tools for text detection and recognition in images.
YOLO	(You Only Look Once) A real-time object detection system known for its speed and efficiency in identifying and localizing multiple objects within an image or video frame.
FastAPI	A modern, high-performance web framework for building APIs with Python, based on standard Python type hints, which makes it easy to use and highly efficient.

1.5.5 Integrations

Table 6: Integrations

Technology	Short Description
Stripe	A technology company that builds economic infrastructure for the internet. It provides APIs for payments, billing, subscriptions, and more.
Google Maps API	A suite of APIs and SDKs that allow developers to embed Google Maps into web and mobile applications, adding features like maps, places, and routing.
Exchange API	Generally refers to an Application Programming Interface that allows programmatic access to a trading exchange for fetching market data, placing orders, and managing accounts (this could be for financial instruments or cryptocurrencies).
Mapbox	A location data platform that provides tools and APIs for creating custom maps, navigation, and location-based services for web and mobile applications.

2 Requirements Specification

2.1 General Requirements

- Platform Compatibility:
 - **Parking Owner Dashboard and Admin Panel:** Web-based applications accessible through standard browsers
 - **Driver Application:** Cross-platform mobile application compatible with both Android and iOS devices
 - **Backend Infrastructure:** Kubernetes-hosted Java microservices architecture
 - **Parking Gate Controller:** Embedded software compatible with microcomputers (e.g., Raspberry Pi)
- System Integration
 - All components must communicate seamlessly through defined APIs
 - Real-time data synchronization across platform components
 - Consistent user experience across all interfaces

2.2 Functional Requirements

1. Parking Owner Dashboard

- Parking management functionality:
 - Add and configure new parking facilities
 - Update parking capacity, operating hours, and categorization tags
 - Configure and adjust tariff structures and pricing
 - See parking sessions and details about this sessions
- Financial management capabilities:
 - View comprehensive session statistics and analytics
 - Monitor revenue streams from parking operations
 - Request and track payouts to designated bank accounts
 - Access transaction history and financial reports
- Facility operations:
 - Manage parking gate controls and access points
 - Monitor customer reviews and feedback

2. Driver Mobile Application

- Parking discovery and management:
 - Interactive map-based parking lot search functionality
 - Vehicle profile management for multiple vehicles
 - Session initiation, monitoring, and payment processing
- Account management:
 - Balance management with top-up capability
 - Historical session tracking and reporting
 - User profile configuration
- Alternative web-based payment option for non-app users

3. Administrative Dashboard

- System maintenance capabilities:
 - Generate and manage API keys for parking facilities
 - Review and moderate parking facility information
 - Tag management for parking categorization
 - User account oversight and moderation
 - Review management and moderation

4. Parking Gate Controller

- Vehicle identification through license plate recognition
- Automated gate operation (opening/closing) based on session validity
- Communication with central system for access validation

2.3 Non-functional Requirements

- Performance
 - Mobile application response time under 2 seconds for all operations
 - Web interfaces to load within 3 seconds under normal network conditions
 - Gate controller to process vehicle identification within 5 seconds
 - System capable of handling concurrent users across all parking facilities
- Security
 - Secure user authentication and authorization

- Encrypted data transmission between all components
 - Payment information security compliant with industry standards
 - Protection against common web vulnerabilities
 - Secure API authentication mechanisms
- Reliability
 - 99.9% uptime for all critical system components
 - Graceful degradation during partial system failures
 - Data backup and recovery mechanisms
 - Fault tolerance for gate controllers to prevent vehicle lockout
- Scalability
 - Architecture capable of accommodating growing number of parking facilities
 - Support for increasing user base without performance degradation
 - Dynamic resource allocation in cloud infrastructure
- Usability
 - Intuitive interfaces requiring minimal training
 - Responsive design for various screen sizes and devices
 - Accessibility compliance for web components
 - Clear error messaging and recovery paths

3 Application Architecture Design

3.1 Overall Architecture Concept

TODO

3.2 Project Structure

TODO

3.3 Application Modules

3.3.1 Application for Parking Owners

TODO

3.3.2 Mobile application for Drivers

TODO

3.3.3 Admin Dashboard

TODO

3.3.4 Parking Gate Controller

TODO

3.3.5 Payment Page for unregistered drivers

TODO

3.4 User Roles

TODO

4 Project Implementation

4.1 Setup & Run Instructions

For comprehensive setup instructions, including prerequisites, installation steps, and guidance on running the application, please refer to [\[Appendix A\]](#).

5 Testing

5.1 Types of tests

TODO

5.2 Running test

TODO

5.3 Testing Frameworks

TODO

6 Security

6.1 Auth & Authentication

TODO

6.2 Data Protection

TODO

6.3 Common Vulnerabilities

TODO

7 Database

7.1 Schema Overview

TODO

7.2 Migrations & Seeding

TODO

8 Summary

TODO

9 Appendices

9.1 Appendix A: Instalation process

Table of Contents

Introduction	2
Setup Backend Manually	4
Building the Mobile Application	16
Parking Gate Controller	20

Introduction

Welcome to the guide for building and using the **Intelligent Parking System**. This application consists of **12 Java Spring microservices**, designed to provide robust and scalable functionality.

Features

- A **mobile application** built using **React Native** and **Expo**.
- Two web dashboards:
 - **Admin Dashboard** (accessible at <http://localhost:5050>)
 - **Parking Management Dashboard** (accessible at <http://localhost:5300>)
- All services are containerized and deployed to **Docker Hub**.

Deployment

The project can be easily launched in a **Docker environment** using a pre-configured `docker-compose` file, located in the root directory of the project.

Prerequisites

- Ensure **Docker CLI** is installed on your system. You can download it from the official Docker website: Docker Desktop (<https://docs.docker.com/desktop/>).

Installation and Startup

To start the application, simply run the following command in your terminal from the root directory:

```
docker compose up -d
```

This will start all services in detached mode.

Accessing the Dashboards

Once the services are up and running, you can access the web dashboards at the following URLs:

- **Admin Dashboard:** <http://localhost:5050> (<http://localhost:5050>)
- **Parking Management Dashboard:** <http://localhost:5300> (<http://localhost:5300>)

Setup Backend Manually

This guide explains how to set up the microservices architecture for the Parking Application on your local development environment. The application consists of multiple interconnected services that collectively provide a comprehensive parking management solution.

Repository Configuration

To use our core library hosted in Azure Artifacts, you need to configure your Maven settings. The settings file is located at `~/.m2/settings.xml` (Unix/macOS) or `C:\Users\<username>\.m2\settings.xml` (Windows). Create or modify this file with the following content:

```
<settings xmlns="http://maven.apache.org/SETTINGS/1.0.0"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/SETTINGS/1.0.0
    https://maven.apache.org/xsd/settings-
1.0.0.xsd">
  <servers>

    <server>
      <id>security-lib</id>
      <username>fabiansprycha</username>

      <password>FK3Q5JukGrDM4Aiy2uvJcOX3fmRl0Q50fJH7qHCb9onD1At6wdvJJQQJ99BDAC
AAAAAAAAAAAAAAASAZD02hHb</password>
    </server>

  </servers>
</settings>
```

⚠ Important: The provided credentials have read-only access. Do not attempt to use them for other purposes.

Starting the Application

Step 1: Start External Services

To start supporting services (PostgreSQL, Kafka, Zookeeper, Redis, etc.), execute the following command:

```
docker compose -f ./docker-compose-manual.yml up -d
```

Step 2: Configure Environment Variables

Use the following environment variables to configure the application. Replace placeholder values (<value>) with your actual settings. **Example Environment Variables:**

```
SPRING_DB_USERNAME: root
SPRING_DB_PASSWORD: root
SPRING_KAFKA_HOST: localhost
SPRING_KAFKA_PORT: 9082
SPRING_DB_HOST: localhost
SPRING_DB_PORT: 6432 # Connected to PgBouncer
SPRING_GATEWAY_HOST: http://localhost:8222
SPRING_OAUTH2_GOOGLE_CLIENT_ID: <Google_Client_ID>
GOOGLE_CLIENT_SECRET: <Google_Client_Secret>
SPRING_JWT_PRIVATE_KEY: |
  { "p": "private-key-part", "kty": "RSA", "n": "public-key-part" }
SPRING_JWT_PUBLIC_KEY: |
  { "kty": "RSA", "e": "AQAB", "n": "public-key-part" }
```

How to set environment variables:

- **Unix/macOS:** Add them to your shell environment:

```
export SPRING_DB_USERNAME=root
export SPRING_DB_PASSWORD=root
# Add other variables here
```

- **Windows** (Command Prompt):

```
set SPRING_DB_USERNAME=root  
set SPRING_DB_PASSWORD=root
```

Alternatively, create a `.env` file in the project root with the above variables.

Service Dependencies

The services must be started in the correct order due to their dependencies:

1. **Config Server** - Central configuration repository
2. **Discovery Service** - Service registry for microservices communication
3. **Security Service** - Authentication and authorization
4. **Gateway** - API Gateway for routing requests
5. All other services - Can be started in any order after the core services

Detailed Setup Instructions

Config Service

This is the backbone of the microservices architecture, providing centralized configuration for all services. **Prerequisites:** Ensure port `8888` is available.

```
cd Config_Server  
maven install  
cd target  
java -jar config-server-0.0.1-SNAPSHOT.jar
```

Verification: Navigate to `http://localhost:8888/actuator/health` to verify the service is running correctly.

Discovery Service

Eureka service registry that allows microservices to discover and communicate with each other.

```
cd Discovery_Service
mvn install
cd target
java -jar discovery-service-0.0.1-SNAPSHOT.jar
```

Verification: Access the Eureka dashboard at <http://localhost:8761> to see registered services.

Security Service

Handles authentication and authorization for the entire application.

```
cd Security_Service
mvn install
cd target
java -jar security-service-0.0.1-SNAPSHOT.jar
```

Environment Variables:

```
SPRING_REDIS_HOST: localhost
SPRING_APPLICATION_NAME: auth-service
SPRING_REDIS_PORT: 6379
ZIPKIN_URL: http://localhost
ZIPKIN_PORT: 9411
SPRING_EUREKA_SERVICE_HOST: localhost
SPRING_EUREKA_SERVICE_PORT: 8761
SPRING_DB_USERNAME: root
SPRING_DB_PASSWORD: root
SPRING_KAFKA_HOST: localhost
SPRING_KAFKA_PORT: 9082
SPRING_DB_HOST: localhost
SPRING_DB_PORT: 6432 # Connected to PgBouncer
SPRING_GATEWAY_HOST: http://localhost:8222
SPRING_OAUTH2_GOOGLE_CLIENT_ID: <Google_Client_ID>
GOOGLE_CLIENT_SECRET: <Google_Client_Secret>
SPRING_JWT_PRIVATE_KEY: |
    { "p": "private-key-part", ... }
```

```
SPRING_JWT_PUBLIC_KEY: |  
  { "kty": "RSA", "e": "AQAB", "n": "public-key-part" }
```

Generating RSA Keys for JWT

To generate the private/public key pair for JWT authentication:

1. Visit a trusted online JWK generator like:
 - MkJose (<https://mkjwk.org/>)
 - Connect2id (<https://connect2id.com/products/nimbus-jose-jwt/generator>)
2. Use the following settings:
 - **Key Type:** RSA
 - **Key Size:** 2048 bits or higher
 - **Key Use:** Signature
 - **Algorithm:** RS256
 - **Key ID:** Generate a random key ID (optional).
3. Generate the key pair.
4. Copy the private and public keys in **JWK** format to the respective environment variables.

Gateway

API Gateway that routes client requests to appropriate microservices.

```
cd Gateway  
mvn install  
cd target  
java -jar gateway-0.0.1-SNAPSHOT.jar
```

Environment Variables:

```
SPRING_REDIS_HOST: localhost
SPRING_APPLICATION_NAME: gateway-service
SPRING_REDIS_PORT: 6379
ZIPKIN_URL: http://localhost
ZIPKIN_PORT: 9411
SPRING_EUREKA_SERVICE_HOST: localhost
SPRING_EUREKA_SERVICE_PORT: 8761
ALLOWED_ORIGINS:
http://localhost:5500,http://localhost:5173,https://localhost:5173,https://marianciuc.works
```

Verification: Check `http://localhost:8222/actuator/routes` to see available routes.

Administration Service

```
cd Administration_Service
maven install
cd target
java -jar config-server-0.0.1-SNAPSHOT.jar
```

Environment Variables:

```
SPRING_REDIS_HOST: localhost
SPRING_APPLICATION_NAME: admin-service
SPRING_REDIS_PORT: 6379
ZIPKIN_URL: http://localhost
ZIPKIN_PORT: 9411
SPRING_EUREKA_SERVICE_HOST: localhost
SPRING_EUREKA_SERVICE_PORT: 8761
SPRING_DB_USERNAME: root
SPRING_DB_PASSWORD: root
SPRING_KAFKA_HOST: localhost
SPRING_KAFKA_PORT: 9082
SPRING_DB_HOST: localhost
SPRING_DB_PORT: 6432 # Connected to PgBouncer
```

Notification Service

Handles notifications via email, SMS, and push notifications.

```
cd Notification_MS
maven install
cd target
java -jar config-server-0.0.1-SNAPSHOT.jar
```

```
SPRING_PORT: 8022
SPRING_REDIS_HOST: localhost
SPRING_APPLICATION_NAME: notification-service
SPRING_REDIS_PORT: 6379
ZIPKIN_URL: http://localhost
ZIPKIN_PORT: 9411
SPRING_EUREKA_SERVICE_HOST: localhost
SPRING_EUREKA_SERVICE_PORT: 8761
SPRING_DB_USERNAME: root
SPRING_DB_PASSWORD: root
SPRING_KAFKA_HOST: localhost
SPRING_KAFKA_PORT: 9082
SPRING_DB_HOST: localhost
SPRING_DB_PORT: 6432 # Connected to PgBouncer
MAIL_HOST: localhost # Connected to mailhog by default
MAIL_PORT: 1025 # Connected to mailhog by default
USE_AUTH: false # Mailhog is not require any authentication
ENABLE_SSL: false
```

Parking Owner Service

Manages parking lot owner accounts and their properties.

```
cd Parking_Owner_MS
maven install
cd target
java -jar config-server-0.0.1-SNAPSHOT.jar
```

```
SPRING_REDIS_HOST: localhost
SPRING_APPLICATION_NAME: owners-service
SPRING_REDIS_PORT: 6379
ZIPKIN_URL: http://localhost
ZIPKIN_PORT: 9411
SPRING_EUREKA_SERVICE_HOST: localhost
SPRING_EUREKA_SERVICE_PORT: 8761
SPRING_DB_USERNAME: root
SPRING_DB_PASSWORD: root
SPRING_KAFKA_HOST: localhost
SPRING_KAFKA_PORT: 9082
SPRING_DB_HOST: localhost
SPRING_DB_PORT: 6432 # Connected to PgBouncer
```

Parking Service

Core service for parking lot management and operations.

```
cd Parking_MS
maven install
cd target
java -jar config-server-0.0.1-SNAPSHOT.jar
```

```
SPRING_REDIS_HOST: localhost
SPRING_APPLICATION_NAME: parking-service
SPRING_REDIS_PORT: 6379
ZIPKIN_URL: http://localhost
ZIPKIN_PORT: 9411
SPRING_EUREKA_SERVICE_HOST: localhost
SPRING_EUREKA_SERVICE_PORT: 8761
SPRING_DB_USERNAME: root
SPRING_DB_PASSWORD: root
SPRING_KAFKA_HOST: localhost
SPRING_KAFKA_PORT: 9082
```

```
SPRING_DB_HOST: localhost
SPRING_DB_PORT: 6432 # Connected to PgBouncer
```

Payment Service

Processes payment transactions for parking sessions.

```
cd Payments_MS
maven install
cd target
java -jar config-server-0.0.1-SNAPSHOT.jar
```

```
SPRING_REDIS_HOST: localhost
SPRING_APPLICATION_NAME: payment-service
SPRING_REDIS_PORT: 6379
ZIPKIN_URL: http://localhost
ZIPKIN_PORT: 9411
SPRING_EUREKA_SERVICE_HOST: localhost
SPRING_EUREKA_SERVICE_PORT: 8761
SPRING_DB_USERNAME: root
SPRING_DB_PASSWORD: root
SPRING_KAFKA_HOST: localhost
SPRING_KAFKA_PORT: 9082
SPRING_DB_HOST: localhost
SPRING_DB_PORT: 6432 # Connected to PgBouncer
STRIPE_SECRET_KEY: sk_...
EXCHANGE_APP_ID: app_id.. # openexchangerates.org app id
STRIPE_PUBLIC_KEY: pk_...
STRIPE_WEBHOOK: webhook
SPRING_PORT: 8064
```

Reviews Service

Handles user reviews and ratings for parking facilities.

```
cd Reviews_MS
maven install
```

```
cd target
java -jar config-server-0.0.1-SNAPSHOT.jar
```

```
SPRING_REDIS_HOST: localhost
SPRING_APPLICATION_NAME: reviews-service
SPRING_REDIS_PORT: 6379
ZIPKIN_URL: http://localhost
ZIPKIN_PORT: 9411
SPRING_EUREKA_SERVICE_HOST: localhost
SPRING_EUREKA_SERVICE_PORT: 8761
SPRING_DB_USERNAME: root
SPRING_DB_PASSWORD: root
SPRING_KAFKA_HOST: localhost
SPRING_KAFKA_PORT: 9082
SPRING_DB_HOST: localhost
SPRING_DB_PORT: 6432 # Connected to PgBouncer
SPRING_PORT: 8064
```

Session Manager Service

Tracks active parking sessions and vehicle entry/exit.

```
cd Session_Manager_MS
maven install
cd target
java -jar config-server-0.0.1-SNAPSHOT.jar
```

```
SPRING_REDIS_HOST: localhost
SPRING_APPLICATION_NAME: session-service
SPRING_REDIS_PORT: 6379
ZIPKIN_URL: http://localhost
ZIPKIN_PORT: 9411
SPRING_EUREKA_SERVICE_HOST: localhost
SPRING_EUREKA_SERVICE_PORT: 8761
SPRING_DB_USERNAME: root
SPRING_DB_PASSWORD: root
```

```
SPRING_KAFKA_HOST: localhost
SPRING_KAFKA_PORT: 9082
SPRING_DB_HOST: localhost
SPRING_DB_PORT: 6432 # Connected to PgBouncer
SPRING_PORT: 8064
```

Tariffs Manager Service

Manages pricing strategies and tariff calculations.

```
cd Tariffs_Manager_MS
maven install
cd target
java -jar config-server-0.0.1-SNAPSHOT.jar
```

```
SPRING_REDIS_HOST: localhost
SPRING_APPLICATION_NAME: tariffs-service
SPRING_REDIS_PORT: 6379
ZIPKIN_URL: http://localhost
ZIPKIN_PORT: 9411
SPRING_EUREKA_SERVICE_HOST: localhost
SPRING_EUREKA_SERVICE_PORT: 8761
SPRING_DB_USERNAME: root
SPRING_DB_PASSWORD: root
SPRING_KAFKA_HOST: localhost
SPRING_KAFKA_PORT: 9082
SPRING_DB_HOST: localhost
SPRING_DB_PORT: 6432 # Connected to PgBouncer
SPRING_PORT: 8064
```

Car Manager Service

The Car Management microservice handles all vehicle-related operations including registration, ownership management, and vehicle data updates. It's designed to work in conjunction with other services in the parking application architecture.

```
cd Car_Manager_MS
maven install
cd target
java -jar config-server-0.0.1-SNAPSHOT.jar
```

```
SPRING_REDIS_HOST: localhost
SPRING_APPLICATION_NAME: vehicle-service
SPRING_REDIS_PORT: 6379
ZIPKIN_URL: http://localhost
ZIPKIN_PORT: 9411
SPRING_EUREKA_SERVICE_HOST: localhost
SPRING_EUREKA_SERVICE_PORT: 8761
SPRING_DB_USERNAME: root
SPRING_DB_PASSWORD: root
SPRING_KAFKA_HOST: localhost
SPRING_KAFKA_PORT: 9082
SPRING_DB_HOST: localhost
SPRING_DB_PORT: 6432 # Connected to PgBouncer
SPRING_PORT: 8064
```

Building the Mobile Application

There are two ways to build and run this application.

1. **Using Expo Go:** This is a quick and straightforward method. However, this approach does not support native functionalities like **maps**, **Stripe integration**, and other custom native modules, which may lead to bugs or limitations.
2. **Native Build:** Fully utilizing the application's features (e.g., maps, Stripe) requires a native build. Below are the detailed instructions for building the application on **Apple devices**.

Prerequisites

Before proceeding, make sure the following tools are installed and available on your system:

1. Node.js (Version 22 or later)

- Download and install from Node.js official website (<https://nodejs.org>).

2. Xcode (Minimum version 16.4)

- Download and install via the Mac App Store.
- Ensure you have at least one virtual device configured in **Xcode Simulator**.

3. Expo CLI

- If not installed globally, install Expo CLI using the command below:

```
npm install -g expo-cli
```

4. Apple Developer Account (Optional, Recommended)

- Having an Apple Developer account will allow configuring signing certificates and deploying to physical devices.

Steps to Build the Application (iOS)

1. Navigate to the Project Directory

Start by opening the terminal and navigating to the root folder of the project:

```
cd driver_mobile
```

2. Install Dependencies

Install all required libraries and dependencies for the project by running:

```
npm install
```



Tip: If you encounter dependency or version issues:

- Run `npm audit` to check for vulnerabilities.
- Use `npm install --legacy-peer-deps` to resolve peer dependency conflicts.

3. Create a Prebuild

The prebuild process generates all necessary native configurations (e.g., **Pods**, Xcode project files).

```
npx expo prebuild --platform ios
```



Note: During the prebuild process:

- Double-check the log for warnings and errors about your native dependencies.
- Ensure you have a stable internet connection for resolving dependencies.

4. Open the Project in Xcode

After the prebuild process completes, open the generated Xcode workspace:

```
open ios/parkingapp.xcworkspace
```

Xcode Adjustments:

1. In Xcode, go to **Target Settings**.
2. Under **Signing & Capabilities**, select your **Team** (developer account).
3. Disable unnecessary capabilities like **Push Notifications** if they are not needed.
4. Review build errors, if any, and check for missing signing certificates.

5. Run the App Locally

Once the configuration in Xcode is complete, you can run the app in development mode using:

```
npx expo run:ios
```

This command launches the app on the chosen iOS simulator (or a connected physical iOS device).

Troubleshooting Tips

1. CocoaPods Issues:

- If you face Pod-related errors, try running:

```
cd ios  
pod install  
cd ..
```

- Ensure you have CocoaPods installed (`sudo gem install cocoapods`).

2. Simulator Not Launching:

- Open Xcode and ensure a virtual device is configured under **Devices and Simulators**.

3. Dependencies Conflicts:

- Clean the npm cache using `npm cache clean --force` and reinstall dependencies.

4. Native Build Errors:

- Verify that native modules like Stripe, Maps, etc., are properly installed. Run `expo doctor` to check for common issues.

Additional Notes

- If you intend to publish the app to the **Apple App Store**, make sure to:
 1. Configure the **App Bundle Identifier** in Xcode.
 2. Use the `expo build:ios` or `eas build` commands for production-ready builds.
- Always test thoroughly on both emulators and physical devices to ensure optimal performance.

With these steps, your mobile application should now be running on iOS devices. If you have any further difficulties, consult the official Expo documentation (<https://docs.expo.dev/>), or feel free to ask for help!

Parking Gate Controller

Parking Gate Controller was created with Python and provides API routes for gate operations such as opening and closing the gate. This project can only be assembled and run on a Raspberry Pi.

Installation and Setup

Follow these steps to set up the Parking Gate Controller on your Raspberry Pi:

1. Connect to Raspberry Pi

To connect, you must be on the same network as the Raspberry Pi:

```
ssh root@raspberrypi.local
```

2. Copy files to Raspberry Pi

Transfer all project files from your local computer to the Raspberry Pi:

```
scp -r /path/to/local/project root@raspberrypi.local:/home/pi/parking-gate-controller
```

3. Go to workspace folder

Navigate to the project directory on the Raspberry Pi:

```
cd /home/pi/parking-gate-controller
```

4. Install Python

Ensure Python is installed on your Raspberry Pi:

```
sudo apt update  
sudo apt install python3 python3-pip
```

5. Create virtual environment

Create and activate a Python virtual environment:

```
python3 -m venv venv
source venv/bin/activate
```

6. Install requirements

Install all required dependencies:

```
pip install -r requirements.txt
```

7. Modify configuration file

Edit the configuration file with your specific settings:

```
nano config.yaml
```

See the Configuration File (["Configuration File" in "Parking Gate Controller"](#)) section for details on available settings.

8. Run application

Start the Parking Gate Controller application:

```
python app.py
```

The controller will now be running and listening for API requests.

Hardware Connections

Connect the following components to the Raspberry Pi's GPIO pins:

LED Connections

- **Red LED:** Connect to GPIO pin 17
 - Positive lead (anode) to GPIO 17 through a 220Ω resistor
 - Negative lead (cathode) to GND
- **Green LED:** Connect to GPIO pin 18

- Positive lead (anode) to GPIO 18 through a 220Ω resistor
- Negative lead (cathode) to GND
- **Yellow LED:** Connect to GPIO pin 27
 - Positive lead (anode) to GPIO 27 through a 220Ω resistor
 - Negative lead (cathode) to GND

Servo Motor Connection

- **Signal wire** (usually orange or yellow): Connect to GPIO pin 12 (PWM pin)
- **Power wire** (usually red): Connect to 5V pin
- **Ground wire** (usually brown or black): Connect to GND

Sound Sensor Connection

- **VCC:** Connect to 3.3V pin
- **GND:** Connect to GND
- **Digital Output (DO):** Connect to GPIO pin 23
- **Analog Output (AO):** Connect to GPIO pin 24 (if using analog readings via ADC)

Note: Always ensure the Raspberry Pi is powered off when connecting or disconnecting hardware components.

Setting Up Ngrok for Remote Access

To make the Parking Gate Controller accessible from the internet, you can use Ngrok to create a secure tunnel to your Raspberry Pi.

1. Install Ngrok

```
# Download Ngrok
wget https://bin.equinox.io/c/bNyj1mQVY4c/ngrok-v3-stable-linux-arm.tgz
```

```
# Extract the archive
tar xvzf ngrok-v3-stable-linux-arm.tgz

# Move ngrok to a directory in your PATH
sudo mv ngrok /usr/local/bin/
```

2. Configure Ngrok

Sign up for a free account at ngrok.com (<https://ngrok.com>) and get your authtoken, then configure it:

```
ngrok config add-authtoken YOUR_AUTH_TOKEN
```

3. Create a startup script

Create a script to run Ngrok as a service:

```
nano start_ngrok.sh
```

Add the following content:

```
#!/bin/bash
ngrok http 5000 --log=stdout > /home/pi/ngrok.log &
```

Make the script executable:

```
chmod +x start_ngrok.sh
```

4. Run Ngrok

```
./start_ngrok.sh
```

5. Get the public URL

```
curl http://localhost:4040/api/tunnels | jq '.tunnels[0].public_url'
```

This will display the public URL you can use to access your Parking Gate Controller.

6. Setup Ngrok as a service (optional)

To ensure Ngrok starts automatically when your Raspberry Pi boots:

```
sudo nano /etc/systemd/system/ngrok.service
```

Add the following content:

```
[Unit]
Description=Ngrok
After=network.service

[Service]
ExecStart=/usr/local/bin/ngrok http 5000
Restart=always
User=pi

[Install]
WantedBy=multi-user.target
```

Enable and start the service:

```
sudo systemctl enable ngrok.service
sudo systemctl start ngrok.service
```

Configuration File

The `config.yaml` file contains all settings for the Parking Gate Controller. Here's an explanation of the available options:

```
# Server Configuration
server:
  host: "0.0.0.0" # Listen on all network interfaces
  port: 5000      # Port for the API server
  debug: false   # Enable/disable debug mode

# GPIO Pin Configuration
gpio:
```

```

red_led: 17      # GPIO pin for red LED
green_led: 18    # GPIO pin for green LED
yellow_led: 27   # GPIO pin for yellow LED
servo: 12        # GPIO pin for servo motor
sound_sensor: 23 # GPIO pin for sound sensor

# Servo Motor Configuration
servo:
    open_angle: 90 # Angle for open gate position (0-180)
    close_angle: 0 # Angle for closed gate position (0-180)
    delay: 0.5     # Delay in seconds for servo movement

# Authentication (if required)
auth:
    enabled: false # Enable/disable authentication
    api_key: ""    # API key for authentication

# Logging Configuration
logging:
    level: "INFO" # Logging level (DEBUG, INFO, WARNING, ERROR,
CRITICAL)
    file: "gate.log" # Log file name
    max_size: 10     # Maximum log file size in MB
    backup_count: 3  # Number of backup logs to keep

# Ngrok Configuration (if using)
ngrok:
    enabled: false # Enable/disable Ngrok integration
    region: "us"   # Ngrok region (us, eu, ap, au, sa, jp, in)

```

Modify these settings according to your specific requirements and hardware configuration.

9.2 API Documentation

9.2.1 Appendix B: Stripe Integration Documentation

Stripe Integration Documentation

This document outlines the integration of the Stripe API into the current system, used as a payment provider. The Stripe API facilitates various payment-related operations, providing a secure and reliable platform for managing account balances. These operations include depositing funds into an account and withdrawing funds to a bank account.

Overview of Integration

- **API Type:** The system utilizes Stripe's RESTful APIs.
- **Authentication:** Configured with role-based access control, providing secure access to authorized users. Publicly available endpoints are secured for unauthorized access where applicable.
- **Use Case:** Useful for managing account balance operations, such as deposits and withdrawals.

API Endpoints

1. Stripe Client Key

The **Stripe Client Key endpoint** provides the public key configured in the system for secure interaction with * *Stripe's API* *. This key is used by the client application to communicate securely with Stripe.

Endpoint Details

HTTP Method	GET
URL	/api/v1/stripe
Authorization	Not required (Publicly accessible)

Example Usage

Request:

```
GET /api/v1/stripe
```

Response:

```
200 OK
Body:
{
  "pk_test_51Habcdefghxxxxx..."
}
```

Error Handling

Error Code	Description
500 INTERNAL SERVER ERROR	Indicates that the <code>stripePublicKey</code> property is not properly configured or cannot be retrieved.

2. Deposit Funds

This endpoint allows authorized users to deposit funds into a specific account. The details, including the amount and currency to be deposited, are provided as part of the request body.

Endpoint Details

HTTP Method	POST
URL	<code>/api/v1/balance/operations/accounts/{accountId}/deposit</code>
Authorization	Requires roles: <code>DRIVER</code> , <code>PARKING_OWNER</code>

Request Parameters

Parameter Type	Parameter Name	Type	Description
Path Variable	accountId	UUID	Unique identifier for the account to deposit funds into.
Request Body	paymentRequest	PaymentRequest (JSON)	Contains details about the payment, including amount and currency.

- **PaymentRequest Schema:**

Field	Type	Description
amount	BigDecimal	The amount to deposit.
currency	String	The currency of the deposit, like USD or EUR.

Response

HTTP Status	Description
200 OK	Returns a confirmation message as a String.
400 BAD REQUEST	Invalid input, e.g., a negative amount.
403 FORBIDDEN	The user lacks the appropriate role.
404 NOT FOUND	The given accountId does not exist.
500 INTERNAL SERVER ERROR	Occurs on unexpected server-side errors.

Example Usage

Request:

```
POST /api/v1/balance/operations/accounts/123e4567-e89b-12d3-a456-426614174000/deposit
Authorization: Bearer <JWT_TOKEN>
Body:
{
  "amount": 150.75,
  "currency": "USD"
}
```

Response:

```
200 OK
"Deposit successfully completed."
```

3. Withdraw Funds

This endpoint enables authorized users to withdraw funds from their account to a specified bank account. The request body provides necessary details, such as the amount and target bank account ID.

Endpoint Details

HTTP Method	POST
URL	<code>/api/v1/balance/operations/accounts/owners/{ownerId}/withdraw</code>
Authorization	Requires roles with withdrawal permissions

Request Parameters

Parameter Type	Parameter Name	Type	Description
Path Variable	ownerId	UUID	The unique identifier of the account owner initiating the withdrawal.
Request Body	withdrawRequest	WithdrawRequest (JSON)	Contains withdrawal details such as amount and bank account ID.

- **WithdrawRequest Schema:**

Field	Type	Description
bankAccountId	UUID	The ID of the bank account to transfer funds to.
amount	BigDecimal	The amount to withdraw.

Response

HTTP Status	Description
200 OK	An empty response body indicating success.
400 BAD REQUEST	Input validation error, e.g., insufficient balance.
403 FORBIDDEN	User lacks proper role permissions to perform this action.
404 NOT FOUND	The ownerId or bankAccountId is not valid.
500 INTERNAL SERVER ERROR	Occurs on unexpected server-side issues.

Example Usage

Request:

```
POST /api/v1/balance/operations/accounts/owners/123e4567-e89b-12d3-a456-426614174000/withdraw
Authorization: Bearer <JWT_TOKEN>
Body:
{
  "bankAccountId": "223e4567-e89b-12d3-a456-426614174001",
  "amount": 300.00
}
```

Response:

```
200 OK
"Withdrawal requested successfully."
```

Notes and Considerations

1. **Security:** All endpoints involving sensitive operations require proper authentication and user roles in the system.
2. **Validation:** Clients should ensure that request parameters meet specified constraints to avoid response errors.
3. **Error Handling:** All errors return descriptive HTTP status codes with possible explanations in the error body.

This documentation should serve as a detailed integration guide for using Stripe API functionalities within the current system.

9.2.2 Appendix C: Email Verification

Email Verification

Overview

The `EmailVerificationController` class handles all functionalities related to email verification for drivers in the system. It provides a RESTful interface for sending email verification codes to drivers and verifying those codes. This controller is exposed under the versioned API path `/api/v1/drivers/email-verification`.

Responsibilities:

1. **Sending Email Verification Code:** Allows authorized drivers to request an email verification code to be sent to their registered email.
2. **Verifying Email:** Provides a mechanism for drivers to verify their email addresses using the code sent.

API Versioning:

- Base Path: `/api/v1`
- Email Verification Endpoint Path: `/drivers/email-verification`

REST Endpoints

1. Verify Email

Endpoint

```
PATCH /api/v1/drivers/email-verification
```

Description

This endpoint verifies an email address using a unique verification code. The code is provided by the driver as a query parameter.

HTTP Method

```
PATCH
```

Request Parameters

- **code** (String, required) A unique verification code sent to the user's email. This code is used to verify the email address.

Authorization

- Drivers must be authenticated to perform this action.
- Uses the `SecurityUtils.getAuthenticatedUserId()` to validate the authenticated driver.

Response

- **Success:**
 - **Status Code:** 200 OK
 - **Body:** No content (void)
- **Error Scenarios:**
 - **400 Bad Request:** If the provided verification code is invalid or expired.
 - **404 Not Found:** If the driver associated with the code cannot be found.
 - **409 Conflict:** If the email has already been verified.

Example Usage

Request:

```
PATCH /api/v1/drivers/email-verification?code=12345
```

Response (Success):

```
HTTP/1.1 200 OK
```

Response (Error - Invalid Validation Code):

```
HTTP/1.1 400 Bad Request
{
```

```
"error": "InvalidValidationCode",  
"message": "The provided validation code is invalid or has expired."  
}
```

2. Send Email Verification Code

Endpoint

POST /api/v1/drivers/email-verification

Description

This endpoint sends a unique email verification code to the email address of the currently authenticated driver.

HTTP Method

POST

Request Body

No request body is required.

Authorization

- Drivers must be authenticated to perform this action.
- Uses `SecurityUtils.getAuthenticatedUserId()` to identify the currently logged-in driver.

Response

- **Success:**
 - **Status Code:** 200 OK
 - **Body:** No content (`void`)
- **Error Scenarios:**
 - **403 Forbidden:** If the user is not authenticated or lacks permissions.
 - **404 Not Found:** If the driver associated with the authenticated ID does not exist.

- **409 Conflict**: If the email has already been verified.

Example Usage

Request:

```
POST /api/v1/drivers/email-verification
Authorization: Bearer <valid-token>
```

Response (Success):

```
HTTP/1.1 200 OK
```

Response (Error - Email Already Verified):

```
HTTP/1.1 409 Conflict
{
  "error": "EmailAlreadyVerified",
  "message": "The email address has already been verified."
}
```

Response Details

Both endpoints return a `ResponseEntity<Void>` object:

- **200 OK**: Indicates the action was completed successfully. There is no body returned.
- **Error Responses**: Return standard HTTP error statuses along with an error response in JSON format.

Example error response:

```
{
  "error": "<ErrorCode>",
  "message": "<Detailed error message>"
}
```

Possible Errors

Below is an exhaustive list of possible exceptions for each endpoint and their associated HTTP status codes.

Verify Email

- **400 Bad Request** - The provided verification code is invalid or expired.
- **404 Not Found** - The driver associated with the verification code cannot be found.
- **409 Conflict** - The email address is already verified.

Send Email Verification Code

- **403 Forbidden** - The user is not authenticated or lacks permissions.
- **404 Not Found** - The authenticated driver does not exist in the system.
- **409 Conflict** - The email address is already verified.

Example Scenarios

1. Verifying an Email (Happy Path)

Request:

```
PATCH /api/v1/drivers/email-verification?code=abc123
```

Response:

```
HTTP/1.1 200 OK
```

2. Sending Email Verification Code

Request:

```
POST /api/v1/drivers/email-verification
```

```
Authorization: Bearer <valid-auth-token>
```

Response:

```
HTTP/1.1 200 OK
```

3. Edge Case - Invalid Code

Request:

```
PATCH /api/v1/drivers/email-verification?code=invalid_code
```

Response:

```
HTTP/1.1 400 Bad Request
{
  "error": "InvalidValidationCode",
  "message": "The provided validation code is invalid or has expired."
}
```

This documentation adheres to RESTful best practices and is structured for easy integration into tools like Swagger or included in project README files for technical reference.

9.2.3 Appendix D: Drivers Manipulations

Drivers Manipulations

The `DriverController` is a REST controller that handles operations related to drivers within the "Drivers Management Service". This controller is responsible for managing drivers' data, including retrieval, search, and updates. It exposes endpoints prefixed with `/api/v1/drivers`, aligning with RESTful design standards and versioning practices. Key responsibilities:

- Retrieve driver details by their unique identifier (`userId`).
- Search drivers based on filtering criteria, with pagination and sorting support.
- Retrieve authenticated driver profile details for the currently logged-in user.
- Update driver details using their unique identifier (`userId`).

Endpoints Overview

1. GET `/api/v1/drivers/{userId}`

Retrieve Driver by ID

Description:

Fetches the details of the driver uniquely identified by the provided `userId`.

Request Details:

- HTTP Method: `GET`
- Path Variable:
 - `userId` (UUID): The unique identifier of the driver to be retrieved.

Access Security:

General access – No specific authentication mechanisms enforced on this endpoint.

Response Details:

- Response Code:

- 200 OK – Driver found.
- 404 Not Found – Driver with the given `userId` does not exist.
- **Response Body:** A JSON representation of the `DriverDto`.

Example Request:

```
GET /api/v1/drivers/e8e70633-e434-48b0-8b0c-9e64c1f50114 HTTP/1.1
Host: api.example.com
```

Example Response:

```
{
  "id": "123e4567-e89b-12d3-a456-426614174000",
  "userId": "e8e70633-e434-48b0-8b0c-9e64c1f50114",
  "profilePicture": "https://example.com/images/user.jpg",
  "firstName": "John",
  "lastName": "Doe",
  "country": "Poland",
  "city": "Gdańsk",
  "email": "johndoe@example.com",
  "phoneNumber": "+48123456789",
  "phoneCode": "+48",
  "birthDate": "1990-01-01",
  "isEmailVerified": true,
  "isPhoneVerified": true,
  "creationDate": "2023-01-01T12:00:00",
  "modificationDate": "2023-02-01T15:30:00"
}
```

Possible Errors:

- 404 Not Found: The driver does not exist for the given `userId`.
- 400 Bad Request: The provided `userId` is malformed.

2. GET /api/v1/drivers/search

Search Drivers

Description:

Searches for drivers based on filtering criteria (e.g., city, email, etc.) with support for pagination and sorting.

Request Details:

- **HTTP Method:** GET
- **Request Parameters:**
 - `size` (Integer, optional): Number of results per page (default: 10).
 - `page` (Integer, optional): Zero-based page index (default: 0).
 - `sort` (String, optional): Sorting order (`asc` or `desc`, default: `asc`).
 - `sortBy` (String, optional): Sorting field (default: `id`).
- **Request Body:**
 - `DriverFilters` (JSON): Filtering criteria for driver attributes.
 - Supported Filter Fields: `firstName`, `lastName`, `country`, `city`, `email`, `phoneNumber`, `phoneCode`, `userId`.

Access Security:

General access – No specific authentication mechanisms enforced.

Response Details:

- **Response Code:**
 - 200 OK – A page of matching drivers found.
 - 400 Bad Request – Invalid request parameters or filter structure.
- **Response Body:** A paginated `Page<DriverDto>` representation.

Example Request:

```
POST /api/v1/drivers/search?page=0&size=5&sort=asc&sortBy=lastName
HTTP/1.1
```

```
Host: api.example.com
```

```
Content-Type: application/json
```

```
{
  "city": "Gdańsk",
  "email": "johndoe@example.com"
}
```

Example Response:

```
{
  "content": [
    {
      "id": "123e4567-e89b-12d3-a456-426614174000",
      "userId": "e8e70633-e434-48b0-8b0c-9e64c1f50114",
      "firstName": "John",
      "lastName": "Doe"
    },
    {
      "id": "123e4567-e89b-12d3-a456-426614174001",
      "userId": "b8e70633-e434-48b0-8b0c-9e64c1f50115",
      "firstName": "Jane",
      "lastName": "Smith"
    }
  ],
  "pageable": {
    "pageNumber": 0,
    "pageSize": 5
  },
  "totalElements": 10,
  "totalPages": 2
}
```

Possible Errors:

- **400 Bad Request:** Malformed filter structure or invalid pagination/sorting parameters.

3. GET /api/v1/drivers

Get Authenticated Driver

Description:

Retrieves the profile of the currently authenticated driver.

Request Details:

- **HTTP Method:** GET
- **Authentication:** Required. User must have the DRIVER role.

Access Security:

- **Pre-Authorization:** Annotated with `@PreAuthorize("hasRole('DRIVER')")`.

Response Details:

- **Response Code:**
 - **200 OK** – Successfully retrieved authenticated driver profile.
 - **403 Forbidden** – User does not have the appropriate role.
- **Response Body:** `DriverDto` object corresponding to the authenticated driver.

Example Request:

```
GET /api/v1/drivers HTTP/1.1
Host: api.example.com
Authorization: Bearer <JWT-Token>
```

Example Response:

```
{
  "id": "123e4567-e89b-12d3-a456-426614174000",
  "userId": "e8e70633-e434-48b0-8b0c-9e64c1f50114",
```

```
"firstName": "John",  
"lastName": "Doe",  
...  
}
```

Possible Errors:

- 403 Forbidden: Not authenticated or missing DRIVER role.

4. PATCH /api/v1/drivers/{userId}

Update Driver

Description:

Updates the details of the driver identified by the given `userId`.

Request Details:

- HTTP Method: PATCH
- Path Variable:
 - `userId` (UUID): The identifier of the driver to be updated.
- Request Body:
 - `DriverDto` (JSON): Contains updated driver details.

Response Details:

- Response Code:
 - 200 OK – Successfully updated the driver details.
 - 404 Not Found – Driver not found.
- Response Body: The updated `DriverDto`.

Example Request:

```
PATCH /api/v1/drivers/e8e70633-e434-48b0-8b0c-9e64c1f50114 HTTP/1.1
```

```
Host: api.example.com
```

```
Content-Type: application/json
```

```
{
  "firstName": "John Updated",
  "lastName": "Doe Updated",
  ...
}
```

Example Response:

```
{
  "id": "123e4567-e89b-12d3-a456-426614174000",
  "userId": "e8e70633-e434-48b0-8b0c-9e64c1f50114",
  "firstName": "John Updated",
  "lastName": "Doe Updated",
  ...
}
```

Possible Errors:

- 404 Not Found: Driver not found for the given `userId`.

Common Error Responses

- 401 Unauthorized: User is not authenticated.
- 403 Forbidden: Authenticated user lacks required permissions.
- 400 Bad Request: Invalid input parameters.

9.2.4 Appendix E: Users Manipulations

Users Manipulations

The `UserController` class provides RESTful endpoints for managing users within the application. It includes functionalities like searching, retrieving user details, blocking/unblocking users, and changing passwords.

Base Endpoint

All user-related APIs are accessed via `/api/v1/users`. The versioning prefix (`/api/v1`) allows for backward compatibility in future updates.

Role in the System

This controller plays a key role in user management, interacting with `UserService` to fetch, update, and administer user data. These endpoints are secured with JWT authentication (`bearerAuth`) and authorization mechanisms.

Endpoints Documentation

1. Search Users

GET /api/v1/users/search

Retrieve a paginated list of users based on optional search criteria such as email, record status, and user type.

Request Parameters:

Parameter	Type	Required	Location	Description
email	String	No	Query	Filter users by email (optional).
page	Integer	No	Query	Page number for pagination (default is 0).
size	Integer	No	Query	Page size for pagination (default is 10).
recordStatus	Enum	No	Query	Filter users by record status (ACTIVE, DELETED, BANNED).
userType	Enum	No	Query	Filter users by their role (DRIVER, PARKING_OWNER, ADMIN).

Response:

- **200 OK:** Returns a paginated `Page<UserDto>` object with user details.
- **400 Bad Request:** Invalid search criteria provided.
- **401 Unauthorized:** The client is not authenticated.

Example Usage:

Request:

```
GET /api/v1/users/search?
email=johndoe@example.com&page=0&size=5&recordStatus=ACTIVE&userType=ADMIN
Authorization: Bearer <JWT_TOKEN>
```

Response:

```
{
  "content": [
    {
      "id": "7bc4ae74-3443-4c53-a82a-8edab9100348",
      "email": "johndoe@example.com",
      "userType": "ADMIN",
      "status": "ACTIVE"
    }
  ],
  "totalPages": 1,
  "totalElements": 1,
  "number": 0,
  "size": 5
}
```

Errors/Edge Cases:

- 400 Bad Request: Invalid `page` or `size` values.
- 401 Unauthorized: Access without a valid JWT token.

2. Get User Details

GET /api/v1/users/{userId}

Retrieve details of a user by their unique ID.

Request Parameters:

Parameter	Type	Required	Location	Description
<code>userId</code>	UUID	Yes	Path	The unique identifier of the user.

Response:

- 200 OK: Returns the `UserDto` with the user's details.
- 404 Not Found: No user found with the provided `userId`.

- **401 Unauthorized:** The client is not authenticated.

Example Usage:

Request:

```
GET /api/v1/users/7bc4ae74-3443-4c53-a82a-8edab9100348
Authorization: Bearer <JWT_TOKEN>
```

Response:

```
{
  "id": "7bc4ae74-3443-4c53-a82a-8edab9100348",
  "email": "johndoe@example.com",
  "userType": "ADMIN",
  "status": "ACTIVE"
}
```

Errors/Edge Cases:

- **404 Not Found:** User ID does not exist.
- **401 Unauthorized:** Access without a valid JWT token.

3. Block a User

PATCH /api/v1/users/{userId}/block

Block a user by their unique ID. **Requires ADMIN privileges.**

Request Parameters:

Parameter	Type	Required	Location	Description
userId	UUID	Yes	Path	The unique identifier of the user to block.

Response:

- **200 OK:** Returns the updated `UserDto` object.
- **403 Forbidden:** The authenticated user does not have the necessary `ADMIN` privileges.
- **404 Not Found:** No user found with the provided `userId`.
- **401 Unauthorized:** The client is not authenticated.

Example Usage:

Request:

```
PATCH /api/v1/users/7bc4ae74-3443-4c53-a82a-8edab9100348/block
Authorization: Bearer <JWT_TOKEN>
```

Response:

```
{
  "id": "7bc4ae74-3443-4c53-a82a-8edab9100348",
  "email": "johndoe@example.com",
  "userType": "ADMIN",
  "status": "BANNED"
}
```

Errors/Edge Cases:

- **403 Forbidden:** User without `ADMIN` privileges attempts this operation.
- **404 Not Found:** User to block not found.
- **401 Unauthorized:** Access without valid token.

4. Unblock a User

PATCH /api/v1/users/{userId}/unblock

Unblock a user by their unique ID. **Requires ADMIN privileges.**

Request Parameters:

Parameter	Type	Required	Location	Description
userId	UUID	Yes	Path	The unique identifier of the user to unblock.

Response:

- **200 OK:** Returns the unblocked user's `UserDto` object.
- **403 Forbidden:** The authenticated user does not have the necessary `ADMIN` privileges.
- **404 Not Found:** User to unblock is not found.
- **401 Unauthorized:** User not authenticated.

Example Usage:

Request:

```
PATCH /api/v1/users/7bc4ae74-3443-4c53-a82a-8edab9100348/unblock
Authorization: Bearer <JWT_TOKEN>
```

Response:

```
{
  "id": "7bc4ae74-3443-4c53-a82a-8edab9100348",
  "email": "johndoe@example.com",
  "userType": "ADMIN",
  "status": "ACTIVE"
}
```

5. Change Password

PUT /api/v1/users

Allows a user to change their password.

Request Body:

Field	Type	Required	Description
oldPassword	String	Yes	The user's current password.
newPassword	String	Yes	The new password to be set.

Response:

- **200 OK:** Password was successfully changed.
- **400 Bad Request:** Validation errors such as invalid passwords.
- **401 Unauthorized:** Access without valid JWT token.

Example Usage:

Request:

```
PUT /api/v1/users
Authorization: Bearer <JWT_TOKEN>
Content-Type: application/json

{
  "oldPassword": "currentPass123",
  "newPassword": "newPass456"
}
```

Authentication and Security

All endpoints require a valid JWT token passed in the `Authorization` header as a bearer token. Some endpoints are restricted to users with `ADMIN` roles.

9.2.5 Appendix F: JWT Public Key

JWT Public Key

The `PublicKeyController` is part of the `pl.edu.zut.app.parking.auth.controllers` package. It is responsible for handling security-related endpoints, particularly for managing cryptographic operations associated with JSON Web Tokens (JWT). This controller provides the public key required to verify JWTs securely in client-server communication.

Base URL

The endpoints under this controller are versioned with `v1` to ensure backward compatibility:

```
/api/v1/security/jwt
```

This API is primarily used by other services, systems, and clients for verifying signatures on JWTs issued by this application.

Endpoints

1. Get JWT Public Key

Description

This endpoint retrieves the public key required to verify cryptographic signatures on JWTs. The public key is used by clients and external systems to validate tokens issued by the application.

Request Details

- **HTTP Method:** `GET`
- **URL:** `/api/v1/security/jwt/public-key`
- **Consumes:** No specific data required in the request.
- **Produces:** `text/plain` (plain text containing the public key)

Annotations

```

@Operation(
    summary = "Get JWT Public Key",
    description = "Retrieves the public key used for cryptographic
operations such as verifying JSON Web Tokens (JWTs).",
    tags = {"JWT", "Security"}
)
@ApiResponses(value = {
    @ApiResponse(responseCode = "200", description = "Successfully
retrieved the public key", content = {@Content(mediaType =
"text/plain")}),
    @ApiResponse(responseCode = "500", description = "Server error while
retrieving the public key", content = @Content)
})
@GetMapping(value = "/public-key", produces =
MediaType.TEXT_PLAIN_VALUE)

```

Response

The API returns the public key as plain text to be used for verifying JWTs.

- **HTTP Status Codes:**
 - **200 OK:** Public key successfully retrieved.
 - **500 Internal Server Error:** An error occurred while retrieving the key.
- **Response Body:**
 - **On Success:** The public key as a **String** (in plain text format).
 - **On Error:** Error details from the server, if applicable.

Authorization

This endpoint does not require authentication because the public key is intended to be accessible by external clients.

Example Usage

Request

```
GET /api/v1/security/jwt/public-key HTTP/1.1
Host: api.example.com
Accept: text/plain
```

Response

```
HTTP/1.1 200 OK
Content-Type: text/plain

-----BEGIN PUBLIC KEY-----
MIIBIjANBgkqhkiG9w0BAQEFAAOCAQ8AMIIBCgKCAQEAnjk2g8N9zKeAsd3...
-----END PUBLIC KEY-----
```

Error Response

If the server encounters any issues while retrieving the public key:

```
HTTP/1.1 500 Internal Server Error
Content-Type: application/json

{
  "timestamp": "2023-10-15T10:05:42.456+00:00",
  "status": 500,
  "error": "Internal Server Error",
  "message": "Could not retrieve public key",
  "path": "/api/v1/security/jwt/public-key"
}
```

Error Scenarios and Edge Cases

Error Code	Description	Possible Causes
500 Internal Server Error	The server encountered an issue while fetching the public key.	- The JWTService implementation failed or threw an exception.
404 Not Found	The endpoint is misconfigured or not found on the server.	- Misalignment in application routing or service mapping during server deployment.

Implementation Notes

- Business Logic:** The method `getPublicKey()` internally calls the `JWTService#getPublicKey` method to fetch the key. This service ensures secure storage and retrieval of the cryptographic materials.
 - The functionality ensures that any downstream system relying on token verification can access the public key.
- API Design Choices:**
 - Versioning:** Versioned under `/api/v1/` to allow upgrades without breaking existing consumers.
 - Plain Text Response:** The public key is returned directly in plain text format to simplify parsing by clients.
- Security:**
 - No authentication is applied because this is a public key.
 - However, this endpoint is sensitive, and more stringent security (e.g., API Gateway rate limiting) may be helpful to prevent misuse (e.g., DDoS).
- Testing and Monitoring:**
 - Ensure regular testing of the endpoint for speed, reliability, and accuracy of the public key retrieval.

- Monitor logs for spikes in requests to ensure no misuse of the system.

9.2.6 Appendix G: Notifications

Notifications

This document provides detailed documentation for the `NotificationController` with added instructions for utilizing WebSocket for real-time notification delivery.

Controller Description

The `NotificationController` is a REST controller that handles CRUD operations for user notifications. It is versioned under `/api/v1/notifications` and serves as part of the system's notification service.

WebSocket Integration: This controller works alongside WebSocket to enable real-time notification delivery to users and groups. Notifications triggered by specific endpoints are sent to WebSocket destinations for immediate updates, allowing clients to react in real-time.

Endpoints Overview

1. Retrieve All Notifications

- **HTTP Method:** `GET`
- **Endpoint:** `/api/v1/notifications`
- **Purpose:** Fetch a paginated list of all notifications.

Request Parameters

- No specific parameters required.

Response

- **Success:** Returns a `200 OK` response with a paginated list of `NotificationDto` objects.

Example Request

```
GET /api/v1/notifications HTTP/1.1
Host: example.com
```

Authorization: Bearer <token>

Example Response

```
{
  "content": [
    {
      "id": "d1223abc-d1df-42e2-bacd-46c08c168159",
      "receiverId": "e311c2f8-f05e-47d3-914c-5bfb35c87944",
      "title": "New Session Started",
      "message": "You started a parking session.",
      "creationDate": "2023-10-05T08:00:00",
      "modificationDate": "2023-10-05T09:00:00"
    }
  ],
  "pageable": {
    "page": 0,
    "size": 10
  }
}
```

Errors/Edge Cases

- **401 Unauthorized:** If the user is not authenticated.
- **500 Internal Server Error:** If the notifications could not be retrieved due to a server error.

2. Mark a Notification as Read

- **HTTP Method:** PUT
- **Endpoint:** /api/v1/notifications/{notificationId}
- **Purpose:** Marks a specific notification as read.

Path Parameters

- **** notificationId ****(UUID): Unique identifier of the notification to be marked as read.

Response

- **Success:** Returns a **200 OK** response with the updated **NotificationDto**.
- **Failure:**
 - **404 Not Found** if the notification does not exist.

Example Request

```
PUT /api/v1/notifications/d1223abc-d1df-42e2-bacd-46c08c168159 HTTP/1.1
Host: example.com
Authorization: Bearer <token>
```

Example Response

```
{
  "id": "d1223abc-d1df-42e2-bacd-46c08c168159",
  "receiverId": "e311c2f8-f05e-47d3-914c-5bfb35c87944",
  "title": "New Session Started",
  "message": "You started a parking session.",
  "creationDate": "2023-10-05T08:00:00",
  "modificationDate": "2023-10-05T09:00:00"
}
```

Errors/Edge Cases

- **404 Not Found:** If the **notificationId** does not exist.
- **401 Unauthorized:** If the user is not authenticated.

3. Delete a Notification

- **HTTP Method:** **DELETE**
- **Endpoint:** **/api/v1/notifications/{notificationId}**
- **Purpose:** Deletes a specific notification.

Path Parameters

- **** notificationId **** (UUID): Unique identifier of the notification to be deleted.

Response

- **Success:** Returns a **200 OK** response with no body.
- **Failure:**
 - **404 Not Found** if the notification does not exist.

Example Request

```
DELETE /api/v1/notifications/d1223abc-d1df-42e2-bacd-46c08c168159
HTTP/1.1
Host: example.com
Authorization: Bearer <token>
```

Example Response

```
HTTP/1.1 200 OK
```

Errors/Edge Cases

- **404 Not Found:** If the **notificationId** does not exist.
- **401 Unauthorized:** If the user is not authenticated.

4. Send a Notification

- **HTTP Method:** **POST**
- **Endpoint:** **/api/v1/notifications/{userId}**
- **Purpose:** Sends a notification to a specific user.

Path Parameters

- **** userId ****(UUID): Unique identifier of the user who will receive the notification.

Request Body

- **** NotificationDto ****:
 - **title** (String, required): The title of the notification.
 - **message** (String, required): The body/message of the notification.

Response

- **Success**: Returns a 200 OK response with no body.

Example Request

```
POST /api/v1/notifications/e311c2f8-f05e-47d3-914c-5bfb35c87944 HTTP/1.1
Host: example.com
Authorization: Bearer <token>
Content-Type: application/json

{
  "title": "Reminder",
  "message": "Your parking session is about to end."
}
```

Example Response

```
HTTP/1.1 200 OK
```

Errors/Edge Cases

- **400 Bad Request**: If the **title** or **message** is missing or invalid.
- **404 Not Found**: If the **userId** does not exist.
- **401 Unauthorized**: If the user is not authenticated.

Real-Time WebSocket Delivery

This endpoint integrates heavily with WebSocket:

1. When a notification is created, it is sent directly to the user's queue.
2. WebSocket ensures that notifications appear in real-time.

WebSocket Example Usage

- **Subscribed Destination:** `/user/{userId}/queue/notifications`
 - Message sent upon notification creation:

```
{
  "id": "d1223abc-d1df-42e2-bacd-46c08c168149",
  "receiverId": "e311c2f8-f05e-47d3-914c-5bfb35c87944",
  "title": "Reminder",
  "message": "Your parking session is about to end.",
  "creationDate": "2023-10-05T08:00:00",
  "modificationDate": "2023-10-05T08:00:00",
  "isRead": false
}
```

General Response Structure

The `ResponseEntity` class is used to structure all responses. It typically includes:

- **Status Code:** Represents success or failure (e.g., 200, 400, 404, etc.).
- **Headers:** Metadata related to the response.
- **Body:** This depends on the endpoint and usually contains the requested data or result.

WebSocket Integration Details

The application uses Spring's `SimpMessagingTemplate` to integrate WebSocket.

Notifications leverage two key destinations:

1. **Personalized Notifications:**

- Subscribed destination: `/user/{userId}/queue/notifications`
- Receives notifications specific to a user (e.g., a reminder, read/unread status updates, deletions).
- Activated by the `sendPersonalNotification` method.

2. Global (Broadcast) Notifications:

- Subscribed destination: `/topic/global`
- Broadcasts system-wide announcements (e.g., maintenance updates).
- Activated by the `sendNotificationToAll` method.

WebSocket Client Setup

To use WebSocket notifications in your application, follow these steps for client-side implementation:

1. Establish WebSocket Connection

Using JavaScript and `STOMP`:

```
const socket = new SockJS('/ws'); // Backend WebSocket endpoint
const stompClient = Stomp.over(socket);

stompClient.connect({}, (frame) => {
  console.log("Connected: " + frame);

  // Subscribe to personalized notifications
  stompClient.subscribe('/user/queue/notifications', (message) => {
    const notification = JSON.parse(message.body);
    console.log("Received notification:", notification);
  });

  // Subscribe to global notifications
  stompClient.subscribe('/topic/global', (message) => {
    const broadcastMessage = JSON.parse(message.body);
    console.log("New global notification:", broadcastMessage);
  });
});
```

```
});  
});
```

2. Handling Notifications

- **Personalized Notifications:**
 - Display or manage notifications intended for a specific user.
 - Update the UI in response to actions such as marking as read or deletion.
- **Global Notifications:**
 - Display critical or system-wide announcements (e.g., maintenance notices).

Error Scenarios (WebSocket-Specific)

- **User Not Subscribed:** If the user has not subscribed to `/user/{userId}/queue/notifications`, they won't receive personal notifications.
- **Connection Issues:** Ensure that the WebSocket connection is properly established before interacting with notifications.
- **Authentication:** WebSocket connections should ensure proper authentication by including any token mechanisms necessary.

With this integration, clients can retrieve notifications via REST and receive them in real-time through WebSocket subscriptions, creating a seamless user experience for notification management and delivery.

Authentication

All endpoints are secured and require a valid `Bearer` token included in the `Authorization` header. Ensure that the user is authenticated to access these APIs.

H I J K L M N O P Q

9.2.7 Appendix H : Vehicle Management

Vehicle Management

The `VehicleController` is a REST controller for managing vehicles in the system. It provides endpoints to create, update, search, and retrieve vehicles. All endpoints are hosted under the `/api/v1/vehicles` base URL, following * *versioning best practices* *. This controller interacts with the **VehicleService** to process and manage vehicle data and ensure secure and efficient data manipulation. It supports pagination for searching vehicles and provides validations for user inputs.

Endpoints

1. Search Vehicles

- **HTTP Method:** GET
- **Endpoint:** `/api/v1/vehicles/search`
- **Description:** Searches for vehicles based on specific filters provided in the request body. Results are paginated with `page` and `size` parameters.

Parameters:

Parameter	Type	Location	Description
<code>page</code>	<code>int</code>	Query Param	The page number (0 by default).
<code>size</code>	<code>int</code>	Query Param	The number of results per page (10 by default).
<code>request</code>	<code>VehicleSearchRequest</code>	Request Body	Contains filters for vehicle search.

Request Example:

```
GET /api/v1/vehicles/search?page=0&size=10
```

```
Content-Type: application/json
```

```
{
  "plateNumber": "ABC-12345",
  "brand": "Toyota",
  "yearOfProduction": 2020
}
```

Response:

HTTP Status	Description	JSON Example
200 OK	Paginated list of vehicles matching the filter.	See response example below.

```
{
  "content": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "brand": "Toyota",
      "model": "Corolla",
      "plateNumber": "ABC-12345",
      "color": "Red",
      "hasOwner": true
    }
  ],
  "pageable": {
    "pageNumber": 0,
    "pageSize": 10
  },
  "totalElements": 1,
  "totalPages": 1
}
```

Possible Errors:

HTTP Status	Error	Description
400 Bad Request	InvalidInputException	Input data validation failed, e.g., invalid JSON or parameter constraints not met.
500 Internal Server Error	VehicleServiceException	Unexpected error in the service layer.

2. Find by Plate Number

- HTTP Method: GET
- Endpoint: /api/v1/vehicles/find-by-plate
- Description: Retrieves a vehicle by its license plate.

Parameters:

Parameter	Type	Location	Description
plate	String	Query Param	The license plate number to query.

Request Example:

```
GET /api/v1/vehicles/find-by-plate?plate=ABC-12345
```

Response:

HTTP Status	Description	JSON Example
200 OK	The vehicle with the specified plate number.	<pre>{ "id": "UUID", "brand": "Toyota", "model": "Corolla", ... }</pre>
404 Not Found	VehicleNotFoundException	No vehicle found with the given plate.

3. Find by Vehicle ID

- HTTP Method: GET
- Endpoint: `/api/v1/vehicles/{vehicleId}`
- Description: Retrieves a vehicle by its unique identifier.

Parameters:

Parameter	Type	Location	Description
vehicleId	UUID	Path Param	The unique ID of the vehicle to query.

Request Example:

```
GET /api/v1/vehicles/550e8400-e29b-41d4-a716-446655440000
```

Response:

HTTP Status	Description	JSON Example
200 OK	The vehicle with the specified ID.	{ "id": "UUID", "brand": "Toyota", "model": "Yaris", ... }
404 Not Found	VehicleNotFoundException	No vehicle found with the given ID.

4. Update a Vehicle

- HTTP Method: PUT
- Endpoint: /api/v1/vehicles/{vehicleId}
- Description: Updates the details of an existing vehicle.

Parameters:

Parameter	Type	Location	Description
vehicleId	UUID	Path Param	The unique ID of the vehicle to update.
vehicleDto	VehicleDto	Request Body	The updated vehicle details.

Request Example:

```
PUT /api/v1/vehicles/550e8400-e29b-41d4-a716-446655440000
Content-Type: application/json

{
  "brand": "Toyota",
  "model": "Camry",
  "plateNumber": "XYZ-67890",
  "color": "Blue"
}
```

Response:

HTTP Status	Description	JSON Example
200 OK	Updated vehicle details.	{ "id": "UUID", "brand": "Toyota", "model": "Camry", ... }
404 Not Found	VehicleNotFoundException	No vehicle found with the given ID.

5. Create a New Vehicle

- HTTP Method: POST
- Endpoint: /api/v1/vehicles
- Description: Creates a new vehicle in the system.

Parameters:

Parameter	Type	Location	Description
vehicleDto	VehicleDto	Request Body	The details of the vehicle to create.
assignOwnership	Boolean	Query Param	Whether to assign ownership to the vehicle. Defaults to false.

Request Example:

```
POST /api/v1/vehicles?assignOwnership=true
Content-Type: application/json
```

```
{
  "brand": "Ford",
  "model": "Fiesta",
  "plateNumber": "FORD-2023",
```

```
"color": "Blue",  
"yearOfProduction": 2023,  
"vin": "1HGCM82633A123456"  
}
```

Response:

HTTP Status	Description	JSON Example
201 Created	The newly created vehicle.	{ "id": "UUID", "brand": "Ford", "model": "Fiesta", ... }
400 Bad Request	InvalidVehicleDataException	Provided data is invalid.
409 Conflict	VehicleAlreadyExistsException	A vehicle with this plate already exists.

Error Responses

Error Code	Description
400 Bad Request	Validation errors in input data.
404 Not Found	Resource not found (e.g., invalid ID or plate).
409 Conflict	Resource conflict (e.g., duplicate plate).
500 Internal	Unexpected server-side error.

9.2.8 Appendix I: Vehicle Ownership Management

Vehicle Ownership Management

The **OwnershipController** is a REST API controller for managing vehicle ownership-related operations in the system. It handles all interactions related to transferring and managing vehicle ownerships between users. This controller provides endpoints under the base URL: `/api/v1/vehicles/{vehicleId}/ownership`.

Responsibilities

- Initiates and manages ownership transfer requests.
- Approves or rejects ownership transfer requests.
- Manages secure ownership transfers with restricted access to specific roles (e.g., admin only).

REST API Endpoints

1. Request Ownership Transfer

Endpoint

```
POST /api/v1/vehicles/{vehicleId}/ownership/transfer/request
```

Description

This endpoint allows a user to initiate a request to transfer the ownership of a vehicle specified by its `vehicleId`.

Request Details

- **Path Parameter:**
 - `vehicleId` (UUID): The ID of the vehicle for which the ownership transfer request is initiated.
- **Request Body:**
 - **Type:** `OwnershipTransferRequest`

- Currently, this object does not have any fields, but it is reserved for extending future requirements.

Response

- **Status:** 200 OK
- **Body:** A Boolean value indicating whether the ownership transfer request was successfully initiated.

Authentication

No role restrictions for this endpoint. Any authenticated user can access it.

Example Request

```
POST /api/v1/vehicles/123e4567-e89b-12d3-a456-426614174000/ownership/transfer/request
Content-Type: application/json
```

Example Response

```
true
```

Error Cases

- 404 Not Found: If no vehicle with the specified `vehicleId` exists.

2. Perform Ownership Transfer

Endpoint

```
POST /api/v1/vehicles/{vehicleId}/ownership/transfer
```

Description

Transfers the ownership of a vehicle to a new owner. This is restricted to users with the `ADMIN` role.

Request Details

- **Path Parameter:**

- `vehicleId` (UUID): The ID of the vehicle whose ownership needs to be transferred.

- **Query Parameters:**

- `newOwnerId` (UUID): The unique ID of the new owner.
- `ownerType` (String): Type of the new owner (`PERSON` or `FAMILY`).

Response

- **Status:** `200 OK`
- **Body:** A `Boolean` value indicating whether the transfer was successfully completed.

Authentication

- `@PreAuthorize("hasRole('ADMIN')")`: Only users with the `ADMIN` role can access this endpoint.

Example Request

```
POST /api/v1/vehicles/123e4567-e89b-12d3-a456-426614174000/ownership/transfer?newOwnerId=456b789c-e89b-56d3-b456-426614174111&ownerType=PERSON
Content-Type: application/json
```

Example Response

```
true
```

Error Cases

- `400 Bad Request`: If the `ownerType` is invalid or missing.
- `404 Not Found`: If no vehicle with the given `vehicleId` exists.
- `403 Forbidden`: If the user does not have the required `ADMIN` role.

3. Accept Ownership Transfer Request

Endpoint

```
PUT /api/v1/vehicles/{vehicleId}/ownership/{requestId}/accept
```

Description

Accepts a pending ownership transfer request for a specific vehicle.

Request Details

- **Path Parameters:**
 - `vehicleId` (UUID): The ID of the vehicle involved in the request.
 - `requestId` (UUID): The unique ID of the ownership transfer request to be accepted.

Response

- **Status:** 200 OK
- **Body:** A `Boolean` value indicating whether the approval was successful.

Authentication

No specific permissions required, but the user must own the vehicle or be authorized to manage its ownership transfers.

Example Request

```
PUT /api/v1/vehicles/123e4567-e89b-12d3-a456-426614174000/ownership/456f7890-e89b-56d3-b456-426614174111/accept
Content-Type: application/json
```

Example Response

```
true
```

Error Cases

- **404 Not Found:** If no ownership request with the specified `requestId` exists for the vehicle.
- **409 Conflict:** If the request is already accepted/rejected.

4. Reject Ownership Transfer Request

Endpoint

```
PUT /api/v1/vehicles/{vehicleId}/ownership/{requestId}/reject
```

Description

Rejects a pending ownership transfer request for a specific vehicle.

Request Details

- **Path Parameters:**
 - `vehicleId` (UUID): The ID of the vehicle involved in the request.
 - `requestId` (UUID): The unique ID of the ownership transfer request to be rejected.

Response

- **Status:** `200 OK`
- **Body:** A `Boolean` value indicating whether the rejection was successful.

Authentication

No specific permissions required, but the user must own the vehicle or be authorized to manage its ownership transfers.

Example Request

```
PUT /api/v1/vehicles/123e4567-e89b-12d3-a456-426614174000/ownership/456f7890-e89b-56d3-b456-426614174111/reject
Content-Type: application/json
```

Example Response

true

Error Cases

- **404 Not Found:** If no ownership request with the specified `requestId` exists for the vehicle.
- **409 Conflict:** If the request was already accepted/rejected.

General Error Responses

- **400 Bad Request:** If any required parameter is missing or invalid.
- **403 Forbidden:** If the user's role or access level does not permit the requested operation.
- **404 Not Found:** If the specified `vehicleId` or `requestId` does not exist in the system.
- **500 Internal Server Error:** If an unexpected error occurs during processing.

9.2.9 Appendix J: Parking Owners Management

Parking Owners Management

The `OwnerController` is a REST controller in the Parking Owners Management System responsible for managing operations related to parking owners. These operations include checking permissions, retrieving owner information, and updating owner details. All endpoints are prefixed with `** /api/v1/owners **`, following the system's versioning conventions.

Endpoints Overview

1. Check Owner Permissions

Endpoint

`GET /api/v1/owners/check-permission`

Description

Validates if the currently authenticated user has the necessary permissions to log into the parking panel.

Authorization

- Requires `LOGIN_IN_PARKING_PANEL` role.
- This endpoint is protected with `@PreAuthorize`.

Response

- **HTTP 200 OK:** If the authenticated user has the required role, an empty response is returned.
- **HTTP 401 Unauthorized:** If the user is not authenticated or doesn't have the required role.

Example Usage

```
GET /api/v1/owners/check-permission HTTP/1.1
Authorization: Bearer <access_token>
```

Response:

HTTP 200 OK

Error Scenarios

- **401 Unauthorized:** The user is not logged in, or `LOGIN_IN_PARKING_PANEL` role is missing.

2. Get Current Parking Owner Details

Endpoint

GET /api/v1/owners/details

Description

Fetches the details of the currently authenticated parking owner.

Request

- No parameters required.

Response

- **HTTP 200 OK:** Returns the parking owner details in the form of a `ParkingOwnerDto` object.
- **HTTP 401 Unauthorized:** If the user is not authenticated.

Response Body Example

```
{
  "id": "123e4567-e89b-12d3-a456-426614174000",
  "firstName": "John",
  "middleName": "D.",
  "lastName": "Doe",
  "NIP": "1234567890",
  "phoneNumber": "123456789",
  "phoneNumberCode": "+1",
  "address": {
```

```
    "country": "USA",
    "city": "New York",
    "street": "1st Avenue",
    "houseNumber": "123",
    "apartmentNumber": "4B",
    "postalCode": "10001"
  },
  "isRegistrationCompleted": true,
  "creationDate": "2023-01-01T12:34:56",
  "modificationDate": "2023-06-01T09:15:00"
}
```

Example Usage

```
GET /api/v1/owners/details HTTP/1.1
Authorization: Bearer <access_token>
```

Error Scenarios

- **401 Unauthorized**: The user is not authenticated.
- **404 Not Found**: If the authenticated user does not exist.

3. Search Parking Owner Details

Endpoint

```
GET /api/v1/owners/details/search
```

Description

Fetches details of a parking owner using one or more query parameters such as **ownerId**, **firstName**, **lastName**, **NIP**, **phoneNumber**, or **countryCode**.

Request Parameters

- **** ownerId ****(UUID): Identifier of the owner to search for.
- **** firstName ****(String): Owner's first name.

- **** lastName ****(String): Owner's last name.
- **** NIP ****(String): Owner's NIP (tax number).
- **** phoneNumber ****(String): Owner's phone number.
- **** countryCode ****(String): Country code of the owner.

Authorization

- Requires **ADMIN** role.
- Protected by **@PreAuthorize**.

Response

- **HTTP 200 OK**: Search operation executed successfully (body purposely empty).
- **HTTP 401 Unauthorized**: If the user is unauthorized.
- **HTTP 403 Forbidden**: If the user does not have **ADMIN** role.
- **HTTP 404 Not Found**: If no results match the search criteria.

4. Update Parking Owner Address

Endpoint

PUT /api/v1/owners/{userId}/address

Description

Updates the address of a parking owner with the given **userId**.

Request Parameters

- **Path Variable:**
 - **userId** (UUID): The unique identifier of the parking owner whose address is being updated.
- **Request Body:**
 - **AddressDto:**

```
{
  "country": "String",
  "city": "String",
  "street": "String",
  "houseNumber": "String",
  "apartmentNumber": "String",
  "postalCode": "String"
}
```

Response

- **HTTP 200 OK:** Address updated successfully.
- **HTTP 400 Bad Request:** The request body contains invalid data.
- **HTTP 404 Not Found:** The `userId` does not exist.

Example Usage

```
PUT /api/v1/owners/123e4567-e89b-12d3-a456-426614174000/address HTTP/1.1
Content-Type: application/json
Authorization: Bearer <access_token>
```

```
{
  "country": "USA",
  "city": "New York",
  "street": "1st Avenue",
  "houseNumber": "123",
  "apartmentNumber": "4B",
  "postalCode": "10001"
}
```

Response:

```
HTTP 200 OK
```

5. Update Parking Owner Details

Endpoint

PUT /api/v1/owners/{userId}/details

Description

Updates the personal details of a parking owner with the given `userId`.

Request Parameters

- **Path Variable:**
 - `** userId **`(UUID): The identifier of the parking owner.
- **Request Body:**
 - **ParkingOwnerDto:**

```
{
  "id": "UUID",
  "firstName": "String",
  "middleName": "String",
  "lastName": "String",
  "NIP": "String",
  "phoneNumber": "String",
  "phoneNumberCode": "String",
  "address": {
    "country": "String",
    "city": "String",
    "street": "String",
    "houseNumber": "String",
    "apartmentNumber": "String",
    "postalCode": "String"
  },
  "isRegistrationCompleted": true,
  "creationDate": "LocalDateTime",
  "modificationDate": "LocalDateTime"
}
```

Response

- **HTTP 200 OK:** Details updated successfully.
- **HTTP 400 Bad Request:** Invalid data provided in the request body.
- **HTTP 404 Not Found:** The `userId` does not exist.

Example Usage

```
PUT /api/v1/owners/123e4567-e89b-12d3-a456-426614174000/details HTTP/1.1
Content-Type: application/json
Authorization: Bearer <access_token>

{
  "id": "123e4567-e89b-12d3-a456-426614174000",
  "firstName": "John",
  "middleName": "D.",
  "lastName": "Doe",
  "NIP": "1234567890",
  "phoneNumber": "123456789",
  "phoneNumberCode": "+1",
  "address": {
    "country": "USA",
    "city": "New York",
    "street": "1st Avenue",
    "houseNumber": "123",
    "apartmentNumber": "4B",
    "postalCode": "10001"
  },
  "isRegistrationCompleted": true,
  "creationDate": "2023-01-01T12:34:56",
  "modificationDate": "2023-06-01T09:15:00"
}
```

Response:

```
HTTP 200 OK
```

Error Handling For All Endpoints

- **401 Unauthorized:** User is not logged in or doesn't have proper credentials.
- **403 Forbidden:** User does not have the required roles for the endpoint.
- **404 Not Found:** Resource (e.g., `userId`) does not exist.
- **400 Bad Request:** Invalid data provided (e.g., malformed JSON, invalid field values).

9.2.10 Appendix K: Parking Registration Process

Parking Owner Registration Process

The `RegistrationController` handles the registration process for parking owners. It is part of a step-by-step registration workflow that allows a parking owner to provide personal data and address information. All endpoints are prefixed with `**`

`/api/v1/owners/registration**`, following the system's versioning conventions.

After filling that data user status will be changed to `registration_complete`.

Endpoints Overview

1. Step 1: Register Personal Data

Endpoint

POST `/api/v1/owners/registration/step/1`

Description

Registers the personal data of the parking owner as part of the first step in the registration process.

Request

Body

The request body must contain valid personal data, structured as:

- **PersonalDataRegistrationRequest:**

```
{
  "firstName": "String",
  "lastName": "String",
  "email": "String",
  "phoneNumber": "String",
```

```
"NIP": "String"
}
```

Field	Type	Required	Description
firstName	String	Yes	First name of the parking owner.
lastName	String	Yes	Last name of the parking owner.
email	String	Yes	Email address of the parking owner.
phoneNumber	String	Yes	Phone number of the parking owner.
NIP	String	Yes	Tax Identification Number (NIP).

Validation

- Fields must not be null or empty.
- email should be a valid email format.

Response

- **HTTP 200 OK:** Indicates that the personal data has been successfully registered.
- **HTTP 400 Bad Request:** If the request body contains invalid or incomplete data.

Example Usage

```
POST /api/v1/owners/registration/step/1 HTTP/1.1
Content-Type: application/json

{
  "firstName": "John",
  "lastName": "Doe",
  "email": "john.doe@example.com",
```

```
"phoneNumber": "123456789",  
"NIP": "1234567890"  
}
```

Response:

HTTP 200 OK

Error Scenarios

- **400 Bad Request:** Occurs when the provided data fails validation (e.g., missing required fields, invalid email format).
- **500 Internal Server Error:** If the service fails to process the data internally.

2. Step 2: Register Address

Endpoint

POST /api/v1/owners/registration/step/2

Description

Registers the address of the parking owner as part of the second step in the registration process.

Request

Body

The request body must contain valid address information, structured as:

- **AddressDto:**

```
{  
  "country": "String",  
  "city": "String",  
  "street": "String",  
  "houseNumber": "String",  
}
```

```

    "apartmentNumber": "String",
    "postalCode": "String"
  }

```

Field	Type	Required	Description
country	String	Yes	The country of the parking owner.
city	String	Yes	The city of the parking owner.
street	String	Yes	The street name of the parking owner.
houseNumber	String	Yes	The house number of the parking owner.
apartmentNumber	String	No	The apartment number of the parking owner (optional).
postalCode	String	Yes	The postal code of the parking owner.

Validation

- Fields must not be null or empty.
- postalCode must follow the specified format for a valid postal code.

Response

- **HTTP 200 OK:** Indicates that the address data has been successfully registered.

- **HTTP 400 Bad Request:** If the request body contains invalid or incomplete data.

Example Usage

```
POST /api/v1/owners/registration/step/2 HTTP/1.1
Content-Type: application/json
```

```
{
  "country": "USA",
  "city": "New York",
  "street": "1st Avenue",
  "houseNumber": "123",
  "apartmentNumber": "4B",
  "postalCode": "10001"
}
```

Response:

```
HTTP 200 OK
```

Error Scenarios

- **400 Bad Request:** Occurs if required fields are missing or validations fail (e.g., invalid postal code format).
- **500 Internal Server Error:** If the service fails to process the data internally.

Error Handling For All Endpoints

The following HTTP statuses may apply to any endpoint in this controller:

- **400 Bad Request:** Indicates that the request is not valid, due to either missing required fields or incorrect data formats.
- **500 Internal Server Error:** Indicates that the server encountered an unexpected condition and was unable to complete the request.

9.2.11 Appendix L: Reviews

Reviews

The ReviewController manages interactions with parking reviews. It provides endpoints to create, find, delete, report reviews, and manage their moderation status.

- **Base Path:** /api/v1/reviews
- **Versioning:** The v1 in the path ensures future compatibility with evolving API versions.
- **Authentication:** Certain endpoints enforce role-based permissions (DRIVER, MODERATOR, ADMIN) for access.

Endpoints Overview

Method	Path	Description	Roles Allowed
GET	/search	Search reviews with filters and pagination.	Public
POST	/	Create a new parking review.	DRIVER
DELETE	/reviewId	Delete a review by its ID.	DRIVER, MODERATOR, ADMIN
POST	/reviewId/report	Mark a review as reported for further review.	DRIVER, MODERATOR, ADMIN
PUT	/reviewId/status	Change the moderation status of a review.	MODERATOR, ADMIN

Endpoint Details

1. Search Reviews

- **Path:** GET /api/v1/reviews/search
- **Description:** Retrieve a paginated and filtered list of reviews.
- **Request Parameters:**

Type	Name	Description	Required	Example
Query	size	Number of reviews per page (default: 10).	No	5
Query	page	Page number (default: 0).	No	0
Query	sort	Field to sort by (date, rating). (default: date).	No	rating
Query	direction	Sorting direction (asc, desc). (default: desc).	No	asc
RequestBody	ReviewDto	Filters such as author, parking, moderation status, rating.	No	{ "rating": 5 }

- **Response:**

Status Code	Response Type	Description
200 OK	Page<ReviewDto>	A list of reviews matching specified criteria.
404 Not Found	Error	If no reviews matched the criteria.

- **Example Request-Response:**

Request	Response
GET /api/v1/reviews/search?size=5&sort=rating&direction=asc	{"content":[{"reviewText":"Great parking","rating":5,"authorId":"1234"}],"pageable":{"size":5}}

2. Create Review

- **Path:** POST /api/v1/reviews
- **Description:** Create a new review.
- **Authentication:** Requires DRIVER role.
- **Request Body:**

Name	Type	Description	Required	Example
reviewText	String	Text of the review.	Yes	"Nice parking area!"
rating	int	Rating (range 1-5).	Yes	5
authorId	UUID	Unique identifier of the review's author.	Yes	"1234-abcd-5678-efgh"
parkingId	UUID	Unique identifier of the parking being reviewed.	Yes	"5678-dcba-1234-hgfe"

- **Response:**

Status Code	Response Type	Description
200 OK	ReviewDto	Details of the created review.
404 Not Found	Error	If the specified parking does not exist.
400 Bad Request	Error	If a review already exists for the parking.

- **Example Request-Response:**

Request	Response
POST /api/v1/reviews	
{ "reviewText": "Good service", "rating": 4, "authorId": "uuid1234", "parkingId": "uuid5678" }	{ "reviewText": "Good service", "rating": 4, "authorId": "uuid1234", "parkingId": "uuid5678", "moderationStatus": "PENDING", "creationDate": "..." }

3. Delete Review

- **Path:** DELETE /api/v1/reviews/{reviewId}
- **Description:** Delete an existing review by its unique identifier.
- **Authentication:** Requires DRIVER, MODERATOR, or ADMIN roles.
- **Request Parameters:**

Type	Name	Description	Required	Example
Path Variable	reviewId	The unique identifier of the review.	Yes	"uuid-1234-abcd-4567"

- **Response:**

Status Code	Response Type	Description
200 OK	Void	If the review was successfully deleted.
404 Not Found	Error	If the review does not exist.
403 Forbidden	Error	If the user is unauthorized to delete the review.

- **Example Request-Response:**

Request	Response
DELETE /api/v1/reviews/uuid-1234-abcd-4567	200 OK

4. Report Review

- **Path:** POST /api/v1/reviews/{reviewId}/report
- **Description:** Mark a review for reporting/multiple moderation.
- **Authentication:** Requires DRIVER, MODERATOR, or ADMIN roles.
- **Request Parameters:**

Type	Name	Description	Required	Example
Path Variable	reviewId	The unique identifier of the review.	Yes	"uuid-5678-dcba-9876"

- **Response:**

Status Code	Response Type	Description
200 OK	Void	If the review was successfully reported.
404 Not Found	Error	If the review does not exist.

5. Change Review Moderation Status

- **Path:** PUT /api/v1/reviews/{reviewId}/status
- **Description:** Update a review's moderation status (APPROVED, REJECTED, or PENDING).
- **Authentication:** Requires MODERATOR or ADMIN roles.
- **Request Parameters:**

Type	Name	Description	Required	Example
Path Variable	reviewId	The unique identifier of the review.	Yes	"uuid-5678-dcba-9876"
Query Parameter	status	The new moderation status.	Yes	"APPROVED"

- **Response:**

Status Code	Response Type	Description
200 OK	Void	If the status was updated successfully.
404 Not Found	Error	If the review ID does not exist.

- **Example Request-Response:**

Request	Response
PUT /api/v1/reviews/uuid-5678-dcba-9876/status?status=APPROVED	200 OK

Error Handling

Error Code	Possible Cause
404 Not Found	Resource does not exist (e.g., invalid review ID).
400 Bad Request	Invalid input or operation (e.g., duplicate review creation).
403 Forbidden	Unauthorized access to secured endpoint.

9.2.12 Appendix M: Registration and Refresh Token

Registration and Refresh Token

`AuthController` is responsible for handling authentication-related operations in the parking system. It provides endpoints for registering users of specific types (e.g., DRIVER, ADMIN) and refreshing JSON Web Tokens (JWTs) for authentication and session management.

Base URL

All endpoints of this controller are versioned under `/api/v1/security`. Versioning ensures backward compatibility as the system evolves. The controller's role is critical in managing user accounts and token-based authentication flows.

Endpoints

1. Register a New User

HTTP Method and Endpoint

- POST `/api/v1/security/register/{userType}`

Functionality

This endpoint is used to register a new user of a specific type (e.g., DRIVER, ADMIN, PARKING_OWNER). It accepts user registration details (e.g., email, password) and issues authentication tokens upon successful registration.

Request Details

- **Path Variable:**
 - `userType` (required):
 - **Type:** String (UserType Enum)
 - **Description:** Specifies the type of user to register. Supported values are:
 - `DRIVER`: Individual seeking parking spaces.
 - `PARKING_OWNER`: Owner of parking facilities.

- **ORGANIZATION_PERSONAL**: Operational personnel.
- **ADMIN**: System administrator.
- **Example**: "DRIVER"
- **Request Body (JSON)**:
 - **Type**: UserRegistrationRequest
 - **Fields**:
 - **email** (required):
 - **Type**: String
 - **Description**: The email address of the user.
 - **Validation Rules**: Must be a valid email.
 - **Example**: "user@example.com"
 - **password** (required):
 - **Type**: String
 - **Description**: Password for the user account.
 - **Validation Rules**:
 - At least 6 characters long.
 - Must include one uppercase letter, one lowercase letter, one digit, and one special character.
 - **Example**: "Password123!"

Response Details

- **Status Codes**:
 - **200 OK**: User successfully registered. Returns authentication tokens.
 - **400 Bad Request**: Invalid request body or invalid userType.

- 401 Unauthorized: Authorization issue.
- **Response Body (JSON):**
 - **Type:** AuthenticationTokens
 - **Fields:**
 - accessToken:
 - **Type:** String
 - **Description:** The issued access token for authenticated user actions.
 - accessTokenExpiry:
 - **Type:** String
 - **Description:** Expiry timestamp of the access token.
 - refreshToken:
 - **Type:** String
 - **Description:** Token used to refresh expired access tokens.
 - refreshTokenExpiry:
 - **Type:** String
 - **Description:** Expiry timestamp of the refresh token.

Example Usage

Request:

```
POST /api/v1/security/register/DRIVER HTTP/1.1
Content-Type: application/json

{
  "email": "driver@example.com",
  "password": "Password123!"
}
```

Response:

```
HTTP/1.1 200 OK
Content-Type: application/json

{
  "accessToken": "eyJhbGciOiJIUzI1NiIsInR5cC...\"",
  "accessTokenExpiry": "2023-12-31T23:59:59",
  "refreshToken": "dGhpcyBpcyBhIHJlZnJlc2ggdG9rZW4=",
  "refreshTokenExpiry": "2024-01-07T23:59:59"
}
```

Errors/Edge Cases:

- **400 Bad Request:**
 - Missing or invalid fields in the request body (e.g., password length too short).
 - Invalid userType.
- **401 Unauthorized:**
 - Client lacks valid authorization details for registration.

2. Refresh JWT Tokens

HTTP Method and Endpoint

- **POST** /api/v1/security/jwt/refresh

Functionality

This endpoint allows authenticated users to refresh their JWT tokens. It validates the existing refresh token and issues new authentication and refresh tokens.

Request Details

- **Headers:**
 - Authorization:

- **Type:** Bearer Token
- **Description:** The refresh token of the user.
- **Precondition:**
 - User must have the role ROLE_REFRESH_TOKEN.
- **Authentication:**
 - This endpoint requires the user to be authenticated.

Response Details

- **Status Codes:**
 - 200 OK: Tokens successfully refreshed.
 - 403 Forbidden: User does not have the ROLE_REFRESH_TOKEN role.
 - 401 Unauthorized: Invalid or missing token.
- **Response Body (JSON):**
 - **Type:** AuthenticationTokens
 - **Fields:**
 - accessToken:
 - **Type:** String
 - **Description:** The new access token issued.
 - accessTokenExpiry:
 - **Type:** String
 - **Description:** Expiry timestamp of the new access token.
 - refreshToken:
 - **Type:** String
 - **Description:** The new refresh token issued.

- `refreshTokenExpiry`:
 - **Type:** `String`
 - **Description:** Expiry timestamp of the new refresh token.

Example Usage

Request:

```
POST /api/v1/security/jwt/refresh HTTP/1.1
Authorization: Bearer <refresh_token>
Content-Type: application/json
```

Response:

```
HTTP/1.1 200 OK
Content-Type: application/json

{
  "accessToken": "eyJhbGciOiJIUzI1NiIsInR5cC...\"",
  "accessTokenExpiry": "2023-12-31T23:59:59",
  "refreshToken": "dGhpcyBpcyBhbm90aGVyIHJlZnJlc2ggdG9rZW4=",
  "refreshTokenExpiry": "2024-01-14T23:59:59"
}
```

Errors/Edge Cases:

- **401 Unauthorized:**
 - Missing or invalid `Bearer Token` in the `Authorization` header.
- **403 Forbidden:**
 - User is authenticated but does not have the `ROLE_REFRESH_TOKEN`.

Common Response Structure

Both endpoints follow the standard Spring `ResponseEntity` structure:

- **Status Code:** Indicates success or failure of the operation.
- **Headers:** Additional response metadata (e.g., authorization).
- **Body:** Primary data content (e.g., authentication tokens or error message).

Notes

- **Security:**
 - Authentication mechanisms like `Bearer Token` and `ROLE_REFRESH_TOKEN` ensure secure operation.
- **Validation:**
 - Field-level validation is implemented using annotations like `@NotEmpty`, `@Size`, and `@Pattern`.

9.2.13 Appendix N: API Keys Management Service

API Keys Management Service

Comprehensive Documentation for ApiKey REST Controller

The `ApiKey` REST Controller is designed to manage **CRUD operations** and additional functionalities for handling API keys in the **Parking Management System**. These keys are critical for authentication and must be included in all client requests using a custom header: `X-API-Key: {apiKeyValue}`.

The controller interacts with the `ApiKey` entity and provides the ability to **create, retrieve, update, revoke, and delete API keys**. Only users with **administrator privileges** can generate API keys. Below, you will find all the necessary specifications and detailed information for implementing and using the available endpoints.

Endpoint Specifications

1. Create a New API Key

- **URL:** `/api/v1/api-keys/generate/{parkingId}`
- **Method:** `POST`
- **Description:** Allows administrators to create a new API key.
- **Path Parameters:**
 - `parkingId`: UUID of the parkingId to associate new key.
 - **Key Format:**
 - `keyValue`: At least 16 characters long, containing letters and digits.
- **Request Body (JSON):**

```
{
  "scope": ["SCOPE_1", "SCOPE_2"]
}
```

- **Response (201 - Created):**

```
{
  "id": "UUID of the API key",
  "keyValue": "unique-key-value",
  "parkingId": "UUID of the parking",
  "scope": ["SCOPE_1", "SCOPE_2"],
  "issuedBy": "UUID of the issuer",
  "revokedBy": null,
  "status": "ACTIVE"
}
```

- **Error Responses:**

- **400 Bad Request:** Invalid request body structure or missing required fields.

2. Fetch API Key Details

- **URL:** /api/v1/api-keys/{id}

- **Method:** GET

- **Description:** Retrieves the details of a specific API key using its unique ID.

- **Path Parameters:**

- **id:** UUID of the API key to fetch information for.

- **Response (200 - Success):**

```
{
  "id": "UUID of the API key",
  "keyValue": "unique-key-value",
  "parkingId": "UUID of the parking",
  "scope": ["SCOPE_1", "SCOPE_2"],
  "issuedBy": "UUID of the issuer",
  "revokedBy": null,
}
```

```
"status": "ACTIVE"
}
```

- **Error Responses:**

- **404 Not Found:** API key with the specified ID does not exist.

3. List All API Keys

- **URL:** /api/v1/api-keys/search
- **Method:** GET
- **Description:** Fetches a paginated list of all registered API keys with optional filters.
- **Query Parameters:**
 - **status** (Optional): Filter results by status (ACTIVE, REVOKED).
 - **parkingId** (Optional): Filtering by parking id
 - **Filters Logic:** All specified filters are applied using AND logic.
 - **page** (Optional): Page number for pagination (default: 1).
 - **size** (Optional): Number of results per page (default: 10).
- **Response (200 - Success):**

```
{
  "content": [
    {
      "id": "UUID of the API key",
      "parkingId": "UUID of the parking",
      "scope": ["SCOPE_1", "SCOPE_2"],
      "status": "ACTIVE"
    }
  ],
  "page": 1,
  "size": 10,
}
```

```
"totalElements": 50
}
```

4. Revoke an API Key

- **URL:** `/api/v1/api-keys/{id}/revoke`
- **Method:** `PUT`
- **Description:** Revokes an active API key, marking it as no longer valid.
- **Path Parameters:**
 - `id`: UUID of the API key to be revoked.

- **Request Body (JSON):**

```
{
  "revokedBy": "UUID of the current session user (filled
automatically)"
}
```

- **Response (200 - Success):**

```
{
  "id": "UUID of the API key",
  "keyValue": "unique-key-value",
  "parkingId": "UUID of the parking",
  "scope": ["SCOPE_1", "SCOPE_2"],
  "issuedBy": "UUID of the issuer",
  "revokedBy": "UUID of revoker",
  "status": "REVOKED"
}
```

- **Error Responses:**
 - **400 Bad Request:** API key is already revoked.
 - **404 Not Found:** API key with the specified ID does not exist.

5. Delete an API Key

- URL: `/api/v1/api-keys/{id}`
- Method: `DELETE`
- Description: Permanently deletes the specified API key from the system.
- Path Parameters:
 - `id`: UUID of the API key to be deleted.
- Response (204 - No Content):
 - No content provided on success.
- Error Responses:
 - 404 Not Found: API key with the specified ID does not exist.

6. Validate API Key

- URL: `/api/v1/api-keys/validate/{keyValue}`
- Method: `GET`
- Description: Validates the provided API key and retrieves its details if valid.
- Path Parameters:
 - `keyValue`: API key to be validated.
- Response (200 - Success):

```
{
  "id": "UUID of the API key",
  "keyValue": "unique-key-value",
  "parkingId": "UUID of the parking",
  "scope": ["SCOPE_1", "SCOPE_2"],
  "issuedBy": "UUID of the issuer",
  "revokedBy": null,
```

```
"status": "ACTIVE"
}
```

- **Error Responses:**

- **404 Not Found:** API key with the specified ID does not exist.

Request Validation Rules

1. **Key Uniqueness:** The `keyValue` field must be globally unique.

2. **Valid UUID Format:**

- Fields such as `parkingId`, `issuedBy`, and `revokedBy` must conform to the UUID standard.
- `revokedBy` field is populated automatically with the session user's UUID; cannot be set manually.

3. **Field Constraints:**

- `scope` field must not be empty and must include at least one valid permission.
- Valid values for `scope`: `['SCOPE_1', 'SCOPE_2']`. The list of permissions is synchronized with the system's database.
- `status` must be one of the accepted values: `ACTIVE`, `INACTIVE`, or `REVOKED`.

Controller Class Outline

The `ApiKeyController` class implements the following endpoints:

- `@PostMapping("/api/v1/api-keys/generate/{parkingId}")`: Create a new API key.
- `@GetMapping("/api-keys/search")`: List all API keys (paginated).
- `@PutMapping("/api-keys/{id}/revoke")`: Revoke an API key by ID.
- `@DeleteMapping("/api-keys/{id}")`: Delete an existing API key by ID.

- `@DeleteMapping("/api-keys/validate/{keyValue}")`: Validate an API key

Example Use Case

Revoking an API Key

1. Client Action:

- The administrator sends a request to revoke an API key with ID 1234.

2. System Validation:

- Confirms the key exists and its current status is ACTIVE.

3. Outcome:

- API key's status is updated to REVOKED.
- The `revokedBy` field is populated automatically with the session user's UUID; cannot be set manually.

Example Request:

```
PUT /api/v1/api-keys/1234/revoke
Content-Type: application/json
```

Example Response:

```
{
  "id": "1234",
  "keyValue": "unique-key-value",
  "parkingId": "UUID of the parking",
  "scope": ["SCOPE_1", "SCOPE_2"],
  "issuedBy": "UUID of the issuer",
  "revokedBy": "admin-uuid",
  "status": "REVOKED"
}
```

Security and Logging

- This API uses secure authentication mechanisms such as tokens or API key validation.
- All sensitive operations (creation, revocation, deletion) should be logged for auditing purposes.

Notes:

1. Proper error handling ensures robust fault tolerance.
2. This API design is compliant with REST principles for scalability and maintainability.

9.2.14 Appendix O: Parking Management System API Documentation

Parking Management System API Documentation

This documentation provides comprehensive guidance for developers to interact with the **Parking Management System** via REST APIs. It includes endpoint descriptions, request/response formats, role-based permission details, and use case scenarios. The goal is to support frontend developers in seamlessly integrating parking management functionalities into their applications.

1. Introduction

The **Parking Management System API** offers secure and role-based access to manage parking lots efficiently. The system is tailored for two primary roles: **Admins** and **Parking Owners**, ensuring that actions are authorized and access to resources remains secure. The API leverages robust authentication mechanisms to protect sensitive data.

Key Features:

- **Parking Lot Management:** Create, update, delete, and approve parking lots.
- **Advanced Search & Filtering:** Query parking data with customizable filters.
- **Tags Management:** Assign, update, and categorize parking lots using tags.
- **Access Control:** Manage inclusivity using whitelists/blacklists.
- **Operational Updates:** Modify parking lot data such as opening hours and availability.

2. Creating a Parking Lot

Purpose

The `POST /api/v1/parking/creation` endpoint allows authorized users to register new parking lots in the system. The new parking lot will remain in a `CREATION_PROCESS` state until explicitly approved.

Required Parameters

To create a parking lot, the following fields must be included in the body of the request:

Field	Type	Description	Required
name	String	Name of the parking lot.	Yes
address.city	String	City where the parking lot is located.	Yes
address.countryCode	String	Country code in ISO format (e.g., "US").	Yes
address.latitude	Float	Latitude of the location.	Yes
address.longitude	Float	Longitude of the location.	Yes
isBelongToAnyInstitution	Boolean	Indicates if owned by an institution.	No
institutionName	String	Specifies institution name if applicable.	Conditional

Workflow

- 1. Input Preparation:** Ensure all required parameters are provided and formatted correctly.
- 2. Send Request:** Submit a **POST** request to **/api/v1/parking/creation** along with the payload.
- 3. Response:** Upon success, a **201 Created** response is returned with parking lot details.

Common Responses:

- **Success:** **201 Created** A new parking lot entity is returned.

- **Permission Denied: 403 Forbidden** The user does not have a **PARKING_OWNER** role.

3. Updating Parking Lot Status

Purpose

Parking lot statuses can transition between **CREATION_PROCESS**, **ACTIVE**, or **REJECTED** to control visibility and access.

Workflow

1. **Endpoint:** Send a **PUT** request to **/api/v1/parking/{parkingId}/status** with the desired status.
2. **Approval:** Only admins are allowed to change statuses.
3. **Response:** Successful requests return an updated status confirmation.

Example Request:

```
{
  "status": "ACTIVE"
}
```

4. Managing Tags

Tags enable categorization of parking lots for enhanced searchability. Admins can manage tags at both the parking lot and system levels.

Example Use-Cases

- Categorizing lots by features such as **Covered**, **Free Wi-Fi**, or **EV Charging**.
- Assigning tags for filtering and search purposes.

Adding Tags to a Parking Lot

Send a **PUT** request to **/api/v1/parking/{parkingId}/tags**:

```
{
  "tags": [
    "tag_id_1",
    "tag_id_2"
  ]
}
```

Creating New Tags

Admins may create new tags using the `POST /api/v1/tags` endpoint. Required fields:

Field	Type	Description	Required
<code>name</code>	<code>String</code>	Name of the tag.	Yes
<code>description</code>	<code>String</code>	Description of the tag.	Yes

Deleting Tags

Issue a `DELETE /api/v1/tags/{tagId}` to remove tags from lots.

5. Deleting Parking Lots

Purpose

Delete unused or irrelevant parking lots using the `DELETE /api/v1/parking/{parkingId}` endpoint.

Restrictions:

- **Role-Based Access:** Only admins or lot owners can perform deletions.
- **Active Usage:** Lots with active sessions cannot be deleted (`409 Conflict`).

6. Search and Filters

Purpose

Enable users to search for parking lots using query parameters or advanced filters.

Query Parameters:

Parameter	Type	Description
page	Integer	Pagination parameter: current page number.
size	Integer	Defines the number of results per page.
includeHidden	Boolean	Includes parking lots with the status <code>CREATION_PROCESS</code> .

Advanced Filters:

Field	Type	Description
name	String	Parking lot name (e.g., "Central Park").
city	String	Lots located in this city (e.g., "New York").
recordStatus	Enum	Active, Deleted, or Hidden (see Enums below).
parkingStatus	Enum	Defines operational parking lot status.

Enumeration Definitions

Enums are used throughout the API to define controlled values.

1. AccessType Enum

Defines who can access a parking lot:

Value	Description
OPEN	Open to any authorized user.
WHITELIST_ONLY	Access restricted to listed users or vehicles.

2. ParkingStatus Enum

Categorizes the current state of the parking lot:

Value	Description
CREATION_PROCESS	Pending approval after creation.
TEMPORARY_CLOSED	Temporarily unavailable.
OPEN	Operational and accepting parkers.
CLOSED	Closed during non-operating hours.

3. ParkingType Enum

Describes the type of parking lot:

Value	Description
OUTDOOR	Open-air parking.
INDOOR	Covered or garage parking.
MULTI_LEVEL	Multi-tiered parking facilities.
STREET	Curbside or public street parking.
RESIDENTIAL	Reserved for residents of a community or building.

4. RecordStatus Enum

Status of parking data records:

Value	Description
ACTIVE	Lot is available for use.
HIDDEN	Lot is hidden from standard search results.
DELETED	Lot is permanently removed from the system.

This guide ensures that developers have a deep understanding of all the critical functionalities supported by the * *Parking Management System API* *. For additional queries or feature enhancements, reach out to the system administrator or API support team.

9.2.15 Appendix P: Authorization and Authentication System Documentation

Authorization and Authentication System Documentation

This document provides comprehensive guidelines for frontend developers on how to implement authorization and authentication in the system using REST APIs. The system relies on **JWT (JSON Web Tokens)** for secure communication. This guide outlines the authorization workflow, available API endpoints, error handling, and example usage to help you integrate seamlessly into the system.

Overview

The authorization system is designed to securely manage user authentication and protect access to resources using tokens. It supports:

- **Credentials-based authentication:** Login using username and password.
- **JWT-based authorization:** Protect API requests using an access token (short-lived).
- **Token refreshing:** Obtain a new access token when the old one expires without re-authentication.

Authorization Workflow

1. **Authentication:** A user logs in with their credentials and receives two tokens:

- **Access Token:** Used for accessing protected resources.
- **Refresh Token:** Used to generate a new access token when the old one expires.

2. **Securing Requests:** The access token is sent in the **Authorization** header for all secure API calls.

3. **Refreshing Tokens:** If an access token expires, the refresh token is used to obtain a new access token without requiring the user to log in again.

API Endpoints

The following API endpoints define the authentication and authorization flow within the system.

1. Login: Token Retrieval

Authenticate the user and retrieve tokens for future requests.

- **Endpoint:** `/api/v1/security/login`
- **Method:** `POST`

Request (Login):

- **Headers:** None
- **Body:**

```
{
  "login": "<string>",
  "password": "<string>"
}
```

Login Response:

- **200 OK (Success):**

```
{
  "accessToken": "<string>",
  "accessTokenExpiry": "<datetime>",
  "refreshToken": "<string>",
  "refreshTokenExpiry": "<datetime>"
}
```

- **401 Unauthorized (Failure):** If provided credentials are invalid.

2. Access Protected Resources

Access secured endpoints by including the access token in the request header.

- **Endpoint:** Varies based on the specific resource (e.g., `/api/secure/resource`)
- **Method:** Depends on the requested resource (e.g., `GET`, `POST`, etc.)

Request (Protected Resource):

- **Headers:**

```
Authorization: Bearer <accessToken>
```

- **Body** (if required by the resource):

```
{
  "examplePayloadField": "<value>"
}
```

Protected Resource Response:

- **200 OK (Success):** Returns the requested resource.
- **Error Responses:**
 - **401 Unauthorized:** Invalid or expired access token.
 - **403 Forbidden:** User lacks the required permissions to access the resource.

3. Token Refresh

Obtain a new access token using the refresh token.

- **Endpoint:** `/api/v1/security/jwt/refresh-token`
- **Method:** `POST`

Request (Token Refresh):

- **Headers:**

Authorization: Bearer <refreshToken>

- **Body:** None

Refresh Token Response:

- **200 OK (Success):**

```
{
  "accessToken": "<string>",
  "accessTokenExpiry": "<datetime>",
  "refreshToken": "<string>",
  "refreshTokenExpiry": "<datetime>"
}
```

- **401 Unauthorized (Failure):** If the refresh token is invalid or expired.

Detailed Authorization Workflow

1. Login: Authorization Process

1. The user sends a **POST** request to `/api/v1/security/login` with their username and password.
2. The system validates the credentials and returns:
 - An **access token**: Used for accessing resources.
 - A **refresh token**: For generating new access tokens when the current one expires.

2. Securing Requests

1. Add the **access token** to the **Authorization** header of every request to secured resources as follows:

```
Authorization: Bearer <accessToken>
```

2. Provide any additional payload required by the specific endpoint.

3. Refreshing Tokens

1. If the system returns a 401 Unauthorized error due to an expired access token:

- Send a POST request to `/api/v1/security/jwt/refresh-token` with the **refresh token** in the `Authorization` header.

2. Upon success, use the newly issued access token for subsequent requests.

3. If the refresh token is invalid or expired, prompt the user to log in again.

Example Usage Flow

1. Login to Get Tokens

Request:

```
POST /api/v1/security/login
Content-Type: application/json

{
  "login": "exampleUser",
  "password": "password123"
}
```

Response:

```
{
  "accessToken": "eyJhbGciOiJIUzI1NiIsInR5cC...\"",
  "refreshToken": "eyJhbGciOiJIUzI1NiIsInR5cC...\"",
  "accessTokenExpiry": "2025-01-04T19:16:07.298428Z",
}
```

```
"refreshTokenExpiry": "2025-01-05T19:11:07.298013Z"
}
```

2. Use Access Token to Fetch a Protected Resource

Request:

```
GET /api/secure/resource
Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5c...
```

Response:

```
{
  "data": "Some protected resource"
}
```

3. Refresh Expired Token

Request:

```
POST /api/v1/security/jwt/refresh-token
Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5c...
```

Response:

```
{
  "accessToken": "eyJhbGciOiJIUzI1NiIsInR5cC...",
  "refreshToken": "eyJhbGciOiJIUzI1NiIsInR5cC...",
  "accessTokenExpiry": "2025-01-04T19:16:07.298428Z",
  "refreshTokenExpiry": "2025-01-05T19:11:07.298013Z"
}
```

Error Handling

Common Errors

Error Code	Description
401 Unauthorized	The token is invalid, expired, or missing.
403 Forbidden	The user does not have permission to access the requested resource.

Resolving Errors

1. 401 Unauthorized:

- Verify that the access token is valid and included in the `Authorization` header.
- If the token is expired, refresh it using the `/api/v1/security/jwt/refresh-token` endpoint.

2. 403 Forbidden:

- Ensure that the user has sufficient permissions for the resource.

This documentation ensures that frontend developers can effectively authenticate and authorize users, protect sensitive endpoints, and handle common error scenarios when integrating with the system.

9.3 Demo of applications

9.3.1 Web Dashboard for Parking owner

TODO

9.3.2 Mobile Application For Drivers

TODO

9.3.3 Admin Panel

TODO

9.3.4 web-page for non-users mobile app

TODO

9.3.5 Parking Gate Controller

TODO